Content

Abstract ..... **I**

Declaration ..... **II**

Acknowledgments ..... **III**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Project Objectives 2

1.4 Scope and Limitations 3

1.5 Report Organization 4

2 Literature Review **5**

2.1 LLMs in Enterprise Applications 5

2.2 Multi-Agent Systems and Orchestration 5

2.3 Code Generation and Text-to-SQL 6

2.4 RAG 7

2.5 Tool-Augmented Language Models 8

2.6 Data Security and Local Deployment 8

3 Requirements Analysis **10**

3.1 Banking Industry Context 10

3.2 Functional Requirements 10

3.3 Non-Functional Requirements 12

3.4 Use Case Analysis 14

4 System Architecture Design **18**

4.1 Five-Layer Architecture Overview 18

4.2 Frontend Architecture 20

4.3 Backend Microservice Architecture 23

4.5 Data Layer Architecture 29

4.6 Technology Stack Rationale 31

5 Core Agent Implementation **34**

5.1 Tool Agent Implementation 34

5.1.1 Architecture and AI Intent Routing 34

5.1.2 Meeting Room Management 36

5.1.3 Schedule Conflict Detection 38

5.1.4 Route Planning 41
5.2 Code Agent Implementation 45
5.2.1 Architecture Overview 45
5.2.2 Python LLM Inference Server Integration 46
5.2.3 Five-Layer SQL Whitelist Validation 47
5.2.4 Metadata Caching and Execution 52
5.3 RAG Agent Implementation 55
5.4 Copilot — A Unified Intelligent Dispatcher Above the Three Agents 58
5.4.1 Design Rationale 58
5.4.2 Architecture Overview 58
5.4.3 Three-Tier Intent Classification 59
5.4.4 Frontend Dispatch and User Experience 60
5.4.5 Multi-Turn Clarification 61
5.4.6 Cross-Page Communication 61
5.4.7 Second-Level Intent Classification in the Tool Agent 61
5.4.8 AI Provider Configuration Unification 62
6 System Integration and Deployment 60
6.1 User Center and RBAC 60
6.2 Task Center and Orchestration 61
6.3 WebSocket Real-Time Communication 62
6.4 Containerized Deployment 63
7 Discussion 65
7.1 Challenges and Resolutions 65
7.2 Comparison with Existing Solutions 66
7.3 Project Limitations 66
8 Conclusion and Future Work 67
8.1 Summary of Achievements 67
8.2 Future Work 67
References 69
Appendix B: API Documentation (Selected Endpoints) 73
Appendix C: User Manual (Summary) 76
Declaration of Individual Contributions 78
Declaration of Individual Contributions 78

Abstract

Large language models (LLMs) are developing rapidly, opening up new paths for enterprise digital transformation. However, applying them in regulated domains such as banking presents significant challenges. For example, generated code may be incorrect, domain knowledge can be outdated or unverifiable, enterprise tools may be invoked unreliably, and reliance on external APIs introduces data sovereignty risks. To solve these issues, this paper presents BankAgent, a multi-agent AI platform designed for banking digital transformation.

The platform integrates three agents: a Code Agent that converts natural language into SQL through a whitelist-validated Python inference pipeline, a Retrieval-Augmented Generation (RAG) Agent that retrieves policy knowledge using Milvus vector indexing and vLLM generation, and a Tool Agent that handles everyday business tasks such as meeting room booking, schedule conflict detection, and route planning. A unified Copilot dispatcher sits above all three agents, classifying user intent from a single natural language interface and routing each request to the appropriate agent.

The platform enforces role-based access control (RBAC) with tiered security permissions and department-level data isolation. Its five-layer microservice architecture spans a Vue 3 frontend, Spring Cloud Gateway, Spring Boot business and AI services, and a multi-engine data layer, all deployed via Docker Compose.

Declaration

This report is our own original work. We completed it independently. It has not been submitted before for academic qualification requirements at the University of Hong Kong or any other institution. All references to other people's work have been clearly cited and listed in the References section. We have read and followed the University's guidelines on academic integrity. We understand that plagiarism means copying someone else's work without giving credit. We also understand that the University takes plagiarism seriously.

Acknowledgments

We would like to extend our sincere gratitude to our supervisor, Dr. Huang, for his guidance, encouragement, and insightful advice throughout this project. His expertise shaped the direction of our work, and his steady support made the entire experience more rewarding. We are also grateful to the School of Computing and Data Science at the University of Hong Kong for providing the resources and learning environment needed to complete this work. We are grateful to all the faculty members and teaching staff. Their lectures in the MSc in Computer Science programme gave us the theoretical and practical grounding that underpinned this project. We would also like to thank our classmates and peers who took part in discussions. Their feedback during the development and testing phases helped us notice edge cases we had missed. Finally, we thank our families for their patience, encouragement, and unwavering support throughout this journey.

1 Introduction

1.1 Background and Motivation

LLMs are built on transformer architectures. They are trained on large sets of text and have pushed artificial intelligence forward. These systems can understand almost every language, create runnable code from a text prompt, and give detailed answers about many field. This is why engineers see them as a key piece of enterprise digital change.

But using LLMs in business banking brings problems that go beyond just linking up the tech. First, SQL that is made by the model for database queries is not always reliable. The code often has mistakes in structure or meaning, especially when the database has a complex layout. Second, AI models have limited knowledge of bank-specific rules. The rules change often, and model weights learned during training cannot reflect updates in policies. Third, bank workers must integrate AI into their daily work, such as room booking, schedule checking, and route planning. However, present AI models are not always good at calling the right tool or reading the right input.

The most serious problem comes from the common industry practice. Many firms rely on outside LLM APIs, like those from big cloud providers. This brings up data safety issues and legal control problems. The Hong Kong Monetary Authority (HKMA) has strict rules about handling customer information. The rules suggest that institutions must keep control over data access and processing. Therefore, sending internal queries or customer data to an outside API for AI reasoning is not safe in terms of compliance. It is against the principle of data sovereignty.

The current work is driven by several linked problems. These are technical precision, keeping domain knowledge correct, stable tool calling, and data control. So, a single multi-agent AI system is clearly needed. This system must work on-site and take a defense-in-depth approach. It must combine secure code generation, local knowledge retrieval, and enterprise tool calling.

1.2 Problem Statement

The present work are facing four major challenges.

Code generation accuracy. Database queries are used in everyday bank work. For example, they are used to get customer data and to file reports required by rules. Natural-language interfaces can make data access much easier. But LLM-written SQL is not always safe. It must be run through careful checks. The model may add syntax faults, wrong column references, or even harmful operations. The system needs to check multiple levels before executing any SQL statement.

Knowledge retrieval. Answers to procedural and policy questions must be verifiable. They must be backed by citations. This is important for bank employees. Finance is a field where rules must be followed and there is zero tolerance for errors. RAG methods are used to fix LLM issues with old or made-up information. But bank documents have security levels. The system must filter retrieval results to match the user's clearance level. Documents must not leak through answers.

Enterprise Tool Integration. Daily bank work uses many tools, including room booking, calendar syncing, and route planning. An AI system must correctly understand user requests. It must find the right intent, extract the right parameters, and run the matching tool. The system should also use large language model routing to pick the best handler for each request.

Data Sovereignty And Access Control. Beyond the risks from outside APIs, banking settings need strict access levels. Different departments handle documents with different security clearances. The rule of least privilege must be enforced at every layer. Small permission models are often too simple for an organization that has many departments and levels of leadership.

1.3 Project Objectives

This work presents BankAgent, a multi-agent AI platform built on on-premise infrastructure and designed for banking operations. The platform delivers three AI agents under a unified Copilot dispatcher, supported by five platform services and one-command containerized deployment. Table 1 summarizes the project objectives.

Table 1: Project Objectives by Platform Layer

Category	Component	Objective
AI Agents	Code Agent	Safely translate natural language questions into executable SQL queries.
	RAG Agent	Retrieve policy knowledge with verifiable citations and clearance based permission filtering.
	Tool Agent	Automate business tasks such as room booking, schedule conflict detection, and route planning through natural language.
Intelligent Dispatch	Copilot	Classify user intent from a single interface and dispatch requests to the appropriate agent.
Platform Services	Auth & RBAC	Enforce JWT authentication with three role tiers and department scoped data access.
	API Gateway	Provide a unified entry point with JWT filtering, rate limiting, and session enforcement.
	Notification System	Manage multi type messages with a five state approval workflow for sensitive operations.
	Document Management	Store and serve documents with tiered security clearance and HITL escalation.
	Task Center	Orchestrate asynchronous agent execution with retry logic and real time progress streaming.
DevOps	Containerized Deploy	Start all services with a single Docker Compose command.

1.4 Scope and Limitations

This project covers the design, implementation, and evaluation of the multi-agent platform described above. The intended users are banking personnel. The platform provides three main functions. It generates and checks SQL queries. It does knowledge retrieval with citations. It calls business tools. The deployment model is local. It uses Docker Compose for container management. The security model includes JWT authentication and RBAC.

But there are still several limits. First, the Code Agent supports only MySQL SQL and relational table structures. It does not work with other database systems or NoSQL interfaces. Second, the RAG Agent needs a pretrained embedding model. We provide a mock

mode for development, but the real accuracy depends on the model you choose. Third, the Tool Agent only supports three business tools at present. Its scope is defined by the current set of enterprise APIs. Fourth, the platform is built for a single deployment and does not yet support Kubernetes or multi-node scaling.

1.5 Report Organization

The rest of this report is arranged like this. Chapter 2 looks at past work on LLM use in business settings, systems with many agents, turning text into SQL, RAG methods, language models with added tools, and on-site use and data safety. Chapter 3 explains what banks need, functional and non-functional requirements, and user case studies. Chapter 4 shows the five-layer system design, the frontend and backend layouts, data layer construction, and the reasons for choosing each technology. Chapter 5 explains in detail how the three main agents are built. Chapter 6 covers the process of putting the system parts together, including user management, task planning, live communication, and deploying with containers. Chapter 7 talks about the project's difficulties, how this work compares to other solutions, and where it falls short. Chapter 8 sums up the work that was finished and suggests what should be looked at next along with future work.

2 Literature Review

2.1 LLMs in Enterprise Applications

Transformer-based LLMs have changed AI research and its uses. Vaswani et al. (2017)[9] introduced the transformer architecture. Since then, this field gain a rapid progress, large models, like GPT-4, Claude, DeepSeek has come out. These models performs very well in understanding natural languages and inference tasks. Its core relies on self attention mechanism, which can capture semantic associations within the scope of long texts. Through this, coherent text will be generated.

In some specialized organizions, LLMs are used to automatically manage knowledge intensive works, including report writing, document abstract generating, programming code making, specialized field questions answering and complex work flow managing. But if the company use ready-made models, they may face lots of problems. Bender et al. (2021) [11] call these models ”Random Parrot”. They believe that models can only learn statistical laws from training data, with no really understanding ability. So they may output seemingly reasonable answers, which is wrong actually. In regulated industries like bank, this is worrying because it may led to legal penalties and economic losses.

Financial services industry has a strict attitude on using AI. Banks must meet the regulation commands. They need to standardize data confidentiality, transaction audit, and explanatory automate decision. The Basel Committee on Banking Supervision has issued guidelines on AI in the financial industry, requiring transparency, resilience testing, and manual supervision of model outputs.

2.2 Multi-Agent Systems and Orchestration

Multi agent systems (MAS) originated from research focus on distributed artificial intelligence. Early systems used rule-based agents, which had fixed interaction rules. As LLMs are becoming more and more popular, a new architecture design based on intelligent agents has emerged. Under this architecture, each agent relies on language models to complete reasoning, planning, and interactive communication between agents.

Wu et al. (2023) proposed AutoGen [4]. This platform allows different LLM-driven agents to communicate, and work together to solve complex questions. AutoGen has a

dynamic conversation machanism, which can allocate each agent a role, so that subtasks can be transferred and peer review can be used to optimize outputs. But this frame is unpredictable and its output may not be reliable, which is unbearable in commercial scenarios. On the opposite, Hong et al. (2024)[5] proposed MetaGPT. This frame embeds Standard Operating Procedures (SOPs) into a multi-agent collaboration system, drawing inspiration from the working mode of factory assembly lines. Virtual roles such as product managers, system architects, development engineers, and testers are set up, and agents collaborate through the transfer of standardized intermediate results.

These frameworks show promising results in research settings. However, there's still a gap between flexible, role-based multi-agent systems and production systems in banking environments. For process standardization, systems must present deterministic behaviors and tight security controls.

2.3 Code Generation and Text-to-SQL

Converting natural language questions into executable SQL statements has been a major challenge in the field of natural language processing for over 20 years. The early solution adopted a rule-based approach, relying on manually constructed grammar rules and domain specific knowledge systems. This type of method is effective in narrow domain scenarios, but it is difficult to adapt to diverse database structures.

The emergence of the Spider dataset is an important development in this field. Yu et al. (2018) [1] constructed this dataset. Spider is a cross domain large-scale test set that includes 10181 query examples from 138 domains and 200 databases. Its core is used to test the model's ability to handle unfamiliar database architectures. This demand is highly compatible with the banking industry scenario: financial databases will be frequently updated due to new product launches, regulatory reporting requirements, and market data changes.

2.4 RAG

RAG fixed a weakness in LLM. LLM relies on the static knowledge stored in its parameters. This knowledge often leads to factual errors. Lewis et al. (2020) [8] first described RAG. The RAG design has added a step of obtaining information from external sources.

This step retrieves relevant paragraphs from the knowledge base. It provides model evidence for generating answers.

A typical RAG workflow consists of three stages. The first step is index construction: source data is segmented into chunks and then converted into a supported representation (embeddings) using an encoder. Next, LLM generates the final output. This design distinguishes language processing capabilities from actual data storage, enabling generated responses to leverage an organization's specific, real-time data.

RAG technology is highly valuable in the banking sector, as regulatory rules, internal policies, and product documentation are typically stored in vast text repositories. Furthermore, this content is frequently updated, making it impossible to integrate directly into model weights. Pipitone and Alami (2024) introduced LegalBench-RAG [2], a benchmark specifically designed to evaluate RAG systems in the legal and financial domains; this benchmark focuses on clause-level precision rather than document-level recall.

2.5 Tool-Augmented Model

LLMs have inherent limitations that prevent them from directly interacting with external systems or data stores. Tool-augmented language models effectively solve this problem by equipping LLMs with the capability to call APIs, query databases, and perform computational tasks. Yao et al. (2023) [10] introduced the ReAct paradigm, whic combines "Reasoning" and "Acting", demonstrates that integrating reasoning steps with actions enables LLMs to use external tools for tasks involving real-time data, mathmatical calculations, and environmental interactions.

In enterprise applications, the scope of tool usage not only consists of standard web searches, but also specialized tools tailored to specific organizational needs, such as scheduling, resource booking, internal data querying, and process automation. The reliability of tool invocation is a critical performance metric, particularly regarding user intent understanding, precise parameter extraction, and error recovery.

2.6 Data Security and Local Deployment

Such tool invocation, however, is often realised through external cloud APIs, which raises additional security concerns. Security is core consideration of enterprises adopting AI

systems. Currently, a common practice involves accessing LLMs via cloud APIs. That is, external endpoints accepts and processes users' queries and context. This method obeys the data governance policies enforced in regulated sectors. As Karras et al. (2025)[6] noted, sensitive information in prompt payloads may be logged by providers, repurposed for model retraining, or leaked through inversion techniques.

In the banking sector, the Hong Kong Monetary Authority (HKMA) requires the implementation of appropriate safeguards to protect customer information, and also requires that relevant institutions retain control over data access and processing. Similar requirements are found in the EU's General Data Protection Regulation (GDPR), the Personal Information Protection Law (PIPL) of mainland China, and banking security regulations in other jurisdictions. Together, these regulatory frameworks require that confidential financial data be processed in controlled environments that provide full auditability, which is a requirement that cannot be met when data is sent to or processed by third-party services.

On-premises deployment faces a significant computational cost challenge in LLM inference, which Kwon et al. (2023) [7] addressed with the PagedAttention mechanism and the vLLM framework. This solution, inspired by operating system virtual memory management, employs specific memory management strategies to effectively alleviate GPU memory bottlenecks.

Karras et al. (2024) [6] provided a comprehensive review of LLM applications in cybersecurity contexts in the big data era. Their analysis synthesized findings from over 100 research papers covering vulnerability detection, incident response, threat intelligence, and secure code generation. Basically, they found that while LLMs show promise in automating many cybersecurity tasks—such as identifying vulnerabilities in source code and generating security patches—current models still struggle with problems like contextual understanding and high false-positive rates in real-world enterprise settings. For this project, these findings reinforce how important it is to adopt a defense-in-depth security architecture instead of relying on LLM reasoning alone.

3 Requirements Analysis

3.1 Banking Industry Context

The system requirements are derived from banking operations. Banking IT environments share four characteristics that drive the platform's requirements. (i) Large relational databases store client profiles, transaction histories, product catalogs, and compliance records. (ii) Policy and procedure repositories span multiple departments, each document carrying a security classification and subject to frequent regulatory updates. (iii) Administrative workflows such as room booking, calendar coordination, travel planning consume substantial staff hours and remain largely manual. (iv) Security and compliance obligations, including those mandated by the HKMA Supervisory Policy Manual (as discussed in Section 2.6), demand role-based access control, department-scoped data isolation, and auditable escalation processes.

Stakeholder analysis identifies five user groups. Business analysts need to query databases without deep SQL knowledge. Compliance officers need to quickly find relevant regulatory clauses with verifiable references. Administrative staff coordinate meetings and schedules across the organization. Departmental administrators manage team membership, document access rights, and approval workflows in their units. System administrators handle platform configuration, manage user permissions, and allocate resources.

3.2 Functional Requirements

The platform's functional requirements are organized by domain. Table 2 summarizes each requirement with its rationale and priority.

Table 2: Functional Requirements

ID	Requirement	Description
FR1	User Authentication & RBAC	Email-format registration with hashed passwords and stateless JWT authentication. Three roles (Admin, Department Admin, End-User) with fine-grained permissions enforced at the gateway level.
FR2	Department Management	Hierarchical department structure with six banking units. Department administrators can manage membership and assign clearance levels (Public, Internal, Confidential).
FR3	Document Management with Security Clearance	Documents stored with department ownership and security level. Access granted only when the user's department and clearance meet the document's requirements. Inaccessible documents trigger a human-in-the-loop escalation pathway.
FR4	Notification System	Multi-type notifications (chat, access request, SQL audit alert) with a five-state state machine. Unread badge and role-specific views provided on the frontend.
FR5	Code Agent — NL2SQL	Natural language to SQL translation with security validation before execution. Schema metadata cached to improve generation accuracy. Human-in-the-loop mode supported: generate, review, then execute.
FR6	RAG Agent — Policy Q&A	Natural language questions answered from organizational documents with verifiable source citations. Retrieval filtered by the user's department and clearance level.
FR7	Tool Agent — Task Automation	Unified natural language interface for meeting room booking, schedule conflict detection, and route planning. Intent recognized automatically and routed to the appropriate handler.
FR8	Admin Resource Management	Role-restricted CRUD endpoints for meeting rooms and schedules. Validation rules enforce required fields, positive capacity, and chronological time ranges.

3.3 Non-Functional Requirements

Table 3 summarizes the non-functional requirements that govern system quality attributes.

Table 3: Non-Functional Requirements

ID	Requirement	Description
NFR1	Data Sovereignty	All core data processing must run within the on-premise environment. Administrators must be able to configure whether AI inference and embedding use local models or external APIs. When external APIs are used, only the minimum necessary payload is transmitted, with no logging or retention by third-party providers.
NFR2	Performance	SQL generation must complete within 3 seconds. Meeting-room and schedule queries must respond within 500 ms. Route planning must return within 2 seconds. WebSocket progress updates must be delivered within 200 ms. Initial page load must not exceed 3 seconds.
NFR3	Security	All traffic must use HTTPS (WSS for WebSocket). Passwords must be hashed with BCrypt. Authentication must be stateless via signed JWT tokens with configurable expiry. Access must be enforced at the method level. SQL injection must be prevented through parameterized queries. Data deletion must be logical rather than physical for auditability.
NFR4	Reliability	The platform must achieve 99.5% uptime during business hours. External service failures must trigger graceful degradation using mock fallbacks rather than hard failures. All task failures must be logged with user context for post-incident analysis.
NFR5	Maintainability	Services must be independently deployable with clearly separated responsibilities. All APIs must return a consistent response wrapper for uniform error handling. Database operations must use type-safe abstractions with SQL logging for debugging.
NFR6	Deployability	The entire platform must start with a single command. Database schema and seed data must be initialized automatically. Service dependencies must be resolved through health-check probes before dependent containers start.

3.4 Use Case Analysis

The platform serves three human actor roles and interfaces with two external systems,

summarized in Table 4.

Table 4: System Actors

Actor	Type	Capabilities
ROLE_ADMIN	Human	Full system access: manage users, roles, departments, clearance levels; configure AI providers and session policies; CRUD meeting rooms and schedules; view all tasks.
ROLE_DEPT_ ADMIN	Human	Department-scoped access: manage membership and clearance levels within own department; create, edit, and delete department documents; review and approve document access escalation requests.
ROLE_USER	Human	Basic access: submit NL2SQL, RAG, and tool tasks through the Copilot or dedicated pages; query own task history; view and request access to documents within own department and clearance level.
AI Provider	External system	Receives natural language prompts for intent classification, SQL generation, and RAG answer generation. Administrators may configure local or cloud providers.
Amap API	External system	Converts addresses to coordinates (geocoding) and computes driving routes for the Tool Agent's route planning capability.

The five core use cases that span these actors are described below.

Table 4: Use Cases

Table 5: Use Cases

ID	Actor	Goal	Agent	Main Flow
UC1	Business analyst (ROLE_USER)	Query database without writing SQL	Code Agent	User enters a natural language question → LLM generates SQL → whitelist validation checks safety → query executes → tabular result returned with column headers and execution time.
UC2	Compliance officer (ROLE_USER)	Find policy clauses with verifiable citations	RAG Agent	User asks a regulatory question → query is embedded → top-k document chunks retrieved from vector store, filtered by department and clearance → LLM generates answer with source citations.
UC3	Administrative staff (ROLE_USER)	Book a meeting room via natural language	Tool Agent	User describes requirements in natural language → intent classified as MEETING_QUERY → parameters extracted (time, capacity, floor) → available rooms queried against bookings → options displayed.
UC4	Administrative staff (ROLE_USER)	Detect schedule conflicts	Tool Agent	User specifies attendees and time range → intent classified as SCHEDULE_CHECK → each attendee's calendar checked for overlapping entries → conflict report returned with suggested alternatives.
UC5	Staff member (ROLE_USER)	Access a document above own clearance	Notification System	User attempts to open a higher-clearance document → access denied per department and clearance rules → user submits escalation request → department administrator reviews and approves → temporary access granted with audit trail.

Figure 1 illustrates the Tool Agent use cases in UML notation, showing the interaction between the three human roles, two external systems, and the three tool capabilities.

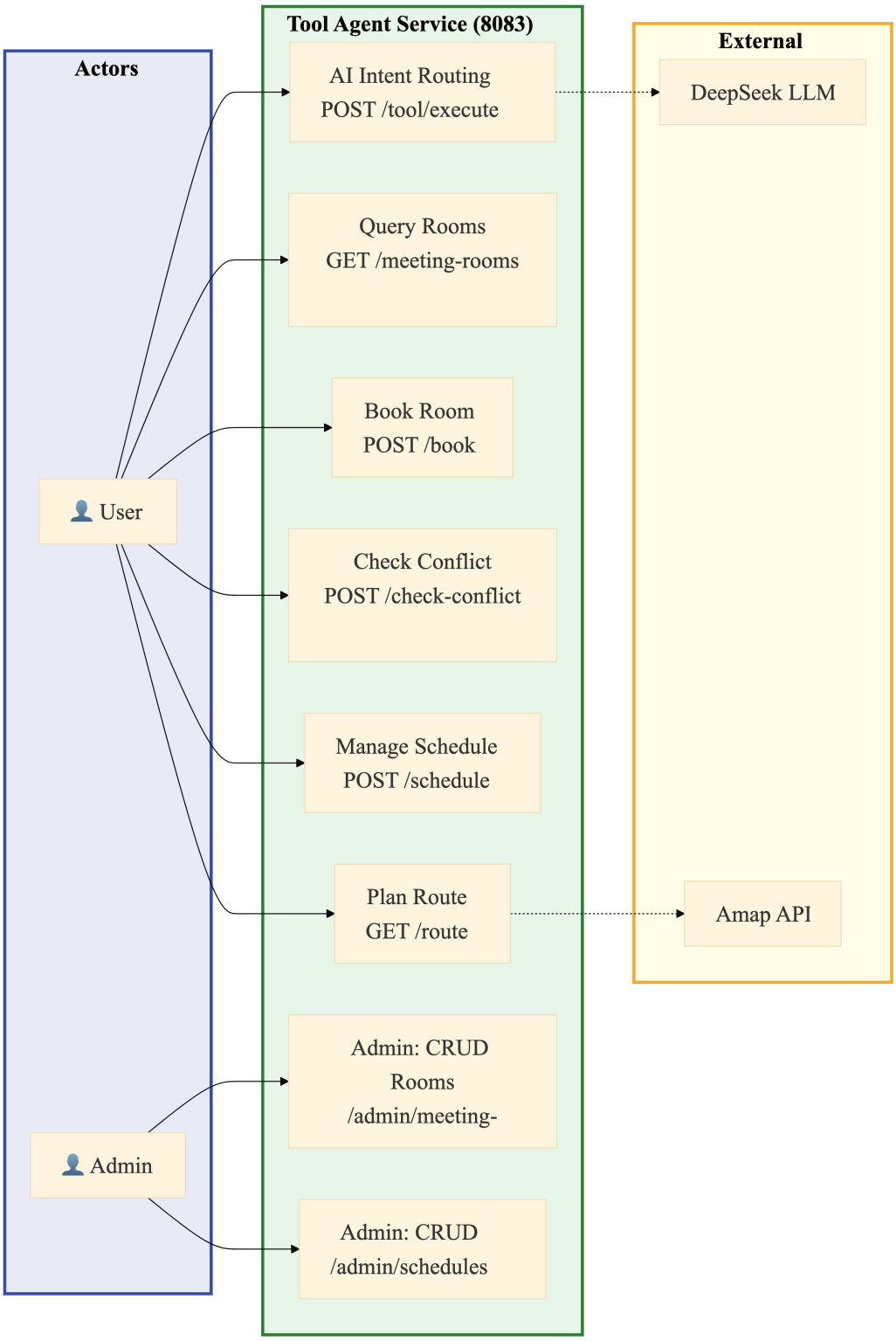

Figure 1: Tool Agent Use Case Diagram — Meeting Room, Schedule Conflict, and Route Planning

4 System Architecture Design

4.1 Five-Layer Architecture Overview

This platform adopts a five-layer architecture with high cohesion and low coupling, implementing one-way dependencies between layers: data is transmitted downwards via RESTful APIs, and long-lived connections are established upwards via WebSockets to proactively push events. This meets the requirements of banking systems for "modularity," "testability," and "independent scalability."

The five-layer structure described above is shown in the diagram below.

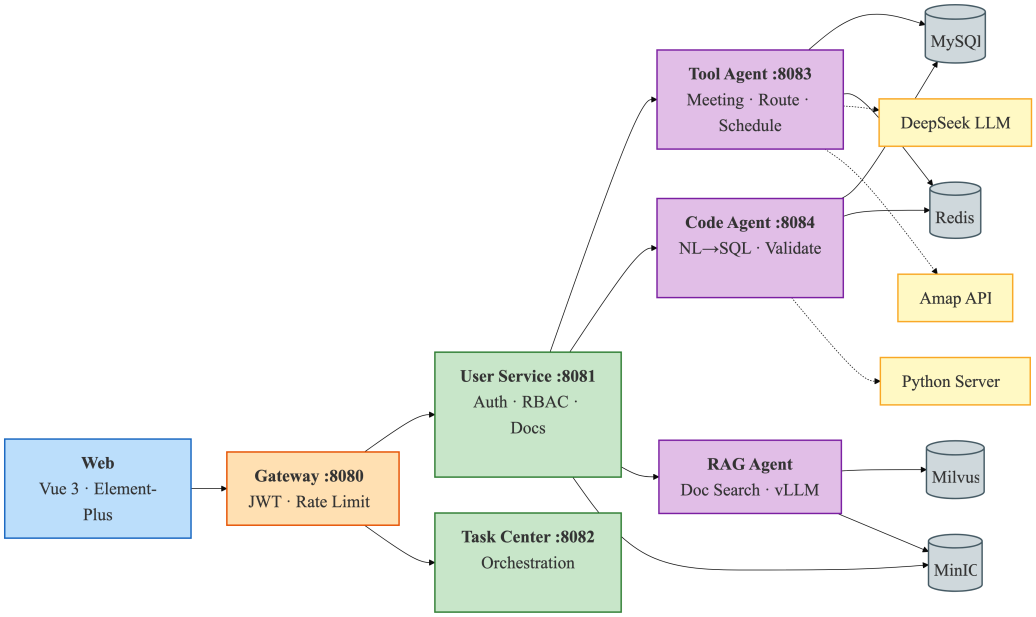

Figure 2: Five-Layer Logical System Architecture — Web, Gateway, Business Service, AI Service, and Data Layers

The five layers, from top to bottom, are as follows:

Layer 1 – Web Presentation Layer. This is the front end of the architecture, using Vue 3 SPA, responsible for user interaction, result display (tables/maps/charts), and real-time progress push notifications. It supports Simplified Chinese, Traditional Chinese, and English language switching.

Layer 2 – the gateway layer. As the system's sole entry point, the gateway layer is responsible for request routing and identity verification. Confidentiality is paramount in banking systems; requests not on the whitelist must be verified using JWT. Upon success-

ful verification, the user's identity and permissions information are extracted and passed to downstream services.

Layer 3 – Business Service Layer. This layer includes user services and a task center. User services handle registration and login, JWT issuance, and role-based access control; the task center handles cross-agent request scheduling and status management.

Layer 4 – AI Service Layer. The AI layer deploys three dedicated agents: the Code Agent handles code generation and security verification before SQL execution; the Tool Agent handles enterprise tool requests, using a large model for intent recognition and parameter extraction; and the RAG Agent combines vector retrieval and generation models to provide policy Q&A with cited sources.

The fifth layer – the data layer. This is the lowest layer of the platform. We adhere to the principle of adapting to local conditions, using MySQL as the single data source to persistently store user, permission, organizational structure, document, and meeting room data; Redis is responsible for caching hot data (such as table structure metadata and schedule conflict detection); Milvus 2.3+ is deployed independently to handle document vectorization storage and semantic retrieval, providing the RAG Agent with similarity search capabilities.

The request lifecycle shown in Figure 3 varies significantly depending on the ingress type. We assume all external traffic first flows into the gateway layer. JWT authentication and rate limiting policies are strictly implemented upfront—the legitimacy of the request and the access rate are determined before routing decisions are made (otherwise, network security issues could easily arise). Only traffic that passes through this filtering layer is allowed to proceed downstream.

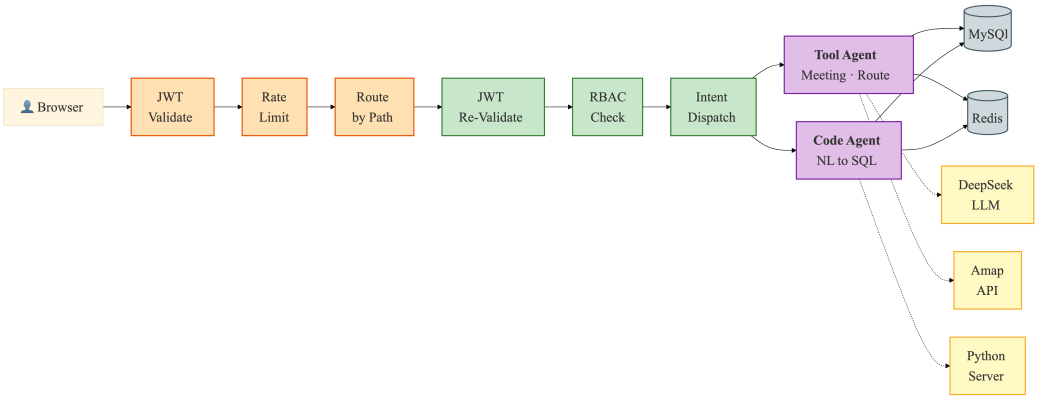

Figure 3: Data Flow Architecture — End-to-End Request Processing Across System Layers

In the Tool agent section, the natural language query (POST /tool/execute) processing path introduces an additional semantic layer. Requests don't directly reach the business logic; instead, they are first sent to DeepSeek for intent "translation"—the model needs to understand whether the user wants to book a meeting room, check a schedule, or plan a route. Only after the intent is clear is the request routed to the corresponding domain module. Meanwhile, the code generation path (POST /code/query) follows a completely different pipeline. The service layer delegates the task to the backend Python inference server. After the SQL is returned, it undergoes a five-layer security check before the Jdbc Template is finally allowed to execute into the database. This link is significantly longer and more likely to become a bottleneck.

The asynchronous task state synchronization mechanism differs from the conventional front-end polling mode. This platform relies on the Web Socket long connection maintained by the gateway proxy to actively push the proxy execution progress to the client, thereby achieving real-time visualization without increasing the request frequency (which will hardly generate any additional latency).

4.2 Frontend Architecture

This project's front-end is based on Vue 3 technology stack, and actual development utilized Composition API and TypeScript. The motivation for introducing TypeScript was straightforward: we wanted to catch type errors during compilation phase, reducing runtime exceptions caused by type mismatches. The logic reuse mechanism provided by Composition API was used simultaneously in Tool, Code, RAG interfaces, and manage-

ment backend—the same set of interaction logic did not need to be implemented repeatedly, thus significantly reducing maintenance cost. We believe that terminal devices used in banking environment is quite complex (desktops, laptops, tablets, and mobile phones are all used, meaning that responsive layout is not an added feature, but a fundamental constraint that must be met.

Route management is handled uniformly by Vue Router, divided into public routes and authenticated routes based on access permissions. Navigation guards intercept requests before route switching, preventing unauthorized requests from entering protected pages. AuthLayout handles the login and registration pages, featuring a slide-in animation during page transitions. MainLayout is the main framework after authentication, with a left sidebar that supports both expanded and collapsed states. When expanded, its 230 pixels wide, and when collapsed, its only 56pixels wide, just enough to accommodate a function icon. The main framework is divided into three areas: agent services (tools, code, RAG) for regular users, a management panel accessible only to ROLE_ADMIN, and system settings. Language switching supports Simplified Chinese, Traditional Chinese, and English(meeting the requirements of Hong Kong banks) .Unread message counts is obtained by polling the /user/notification/unread-count interface every 10 seconds and displayed as badges. The user avatar dropdown menu includes a personal information entry and a logout function.

For state management, we chose Pinia and split it into two Store based on their responsibilities. The userStore is responsible for maintaining login state. JWT tokens is persistently stored in localStorage. userInfo records user roles, permissions, departments, and security levels. External call layers interact with it through the login, logout, and getUserInfo methods. The taskStore tracks the agent task lifecycle. The Task structure includes fields for id, type (code/rag/tool), status (pending/running/completed/failed), progress (0–100), result, and createTime. Status changes are handled through addTask and updateTask.

HTTP requests are uniformly encapsulated in utils/request.ts. The Axios instance is preconfigured with basic URLs and interceptors. Server-side response are uniformly packaged into a Result structure with the fields code, msg, and data. This structure is convenient for front-end response processing. WebSocket communication is handled within utils/websocket.ts, using the wsClient singleton pattern to manage the connection lifecycle. It automatically reconnects after a connection interruption, attempting a maximum of

3 times with a 2-second interval (otherwise, interaction latency would increase. Messages are handled according to four event categories: open, message, close, and error.

Map rendering is handled by the MapContainer.vue component. This component dynamically injects the Amap JS API v2.0, carrying the API key and security code. The map expands from the starting coordinates, with the route drawn as an indigo (#4f46e5) polyline, 6 pixel wide. Custom icons in green and red are used for the start and end points respectively (we've compared them, and color differences improve visibility). The easeOutQuad easing function is used for navigation transitions, making highlight transitions smoother. Full-screen loading feedback is implemented by AgentThinking.vue, with a Sparkles icon placed in center of the screen and a carousel displaying the thinking state text below, the content sourced from a configurable array. The progress bar uses a gradient design, and the progress value will never reach 100% during the loading process. It will fill up instantly when the task is actually completed. This detail is to avoid the user feeling that they are "stuck in the last little segment".

Table 6 summarizes the frontend technology stack.

Table 6: Frontend Technology Stack Specifications

Component	Technology	Utility & Responsibility
View Layer	Vue 3 + TypeScript	Composition API with strict type safety
State Management	Pinia	userStore (JWT persistence), taskStore (task lifecycle)
UI Components	Element-Plus	Enterprise forms, tables, dialogs, navigation elements
Visualization	ECharts 5	Rendering of data charts and SQL execution result analytics
Build Tool	Vite 4	Sub-second HMR during development, Rollup-based production bundling
HTTP Client	Axios + Interceptors	JWT auto-attach, 401 auto-logout, unified error handling
Real-Time	WebSocket Client	Singleton with auto-reconnect (3 attempts, 2s delay)
Internationalization	Vue I18n	zh-CN, zh-TW, en locales with Element Plus sync
Map Visualization	Amap JS API v2.0	Dynamic SDK loading, route polyline, custom markers

4.3 Backend Microservice Architecture

The backend consists of independent Spring Boot micro service, each handling a specific business capability and communicating through the Gateway layer. This decomposition allows independent development, testing, deployment, and scaling of each service, while the Gateway provides a single API entry point for the frontend.

Gateway Service (port 8080). Built on Spring Cloud Gateway 3.1.9, this service provides dynamic routing based on URL path predicates. The JwtAuthGlobalFilter (a GlobalFilter with order=-100)validates tokens for non-whitelist paths. It forwards identity information to downstream services through HTTP headers (X-User-Id, X-Username, X-User-Roles, X-User-Permissions, X-User-Dept-Id, X-User-Security-Level). Single-session enforcement is used. This means when someone logs in again, the old token is deleted from Redis. IP-based rate limiting is also configured: requests sent from the same IP address cannot exceed 100 per second (the burst capacity is 200, returning HTTP 429 upon exceeding). The gateway is the only container whose port is exposed to the outside; all other services are only accessible to each other through the internal Docker network.

User Service (port 8081). This is the largest backend service, covering the complete user lifecycle. Spring Security 6.x is configured with stateless session management, CSRF disabled, and a JwtAuthenticationFilter that extracts the token from the Authorization header as the primary authentication mechanism. A dual-mode login controller accepts either password or verification code credentials, with the token signer using HMAC-SHA256 (the key is injected via environment variable). BCryptPassword Encoder (strength factor 10) handles password storage and verification. Three-tier RBAC is implemented by granting role-specific permissions to an endpoint. The access model to system documents is based on a two-dimensional matrix: clearance level (1-3) and department membership. In the notification subsystem, five state flows support threaded conversations through parent_id and thread_id fields. The email verification subsystem uses Resend HTTP API with HTML formatted templates.

Tool Agent Service (port 8083). The most developed AI service, it implements the Tool Agent with two controller groups. The ToolController (/tool/*) exposes 11 endpoints for end-user tool operations including room queries and reservations, CRUD for personal schedules, conflict detection, route planning, and a unified /tool/execute natural language AI entry point. The MeetingRoomAdminController (/tool/admin/*) exposes CRUD endpoints for administrators—room and schedule management. The AI intent routing module

integrates with DeepSeek API (via iFlytek MaaS) for intent classification, routing requests to MeetingHandler, ScheduleHandler, or RouteHandler accordingly, with MockDetect-Fallback providing graceful degradation when the API is unavailable.

Code Agent Service (port 8084). Implements the NL2SQL pipeline. The OnnxCodeGenerationService (enabled through the code-agent.onnx.enabled=true property) is the sole CodeGenerationService implementation, acting as an HTTP client proxy to the Python inference server. It is responsible for constructing the prompt (including database schema information), forwarding the natural language question, and extracting the generated SQL from the response. The Sql Validation Service performs five-layer whitelist checking. The MetadataCacheManager periodically refreshes the schema metadata cache from information_schema to Redis, so the LLM always has an up-to-date view of the table structure.

Task Service (port 8082). Currently scaffolded with a basic POM configuration and TaskApplication main class. In the current architecture, task orchestration is distributed between the Gateway (request routing) and each agent service (autonomous execution). The separate task service is planned for centralizing asynchronous task tracking, progress management, and cross-agent workflow coordination in future phases.

Table 7 summarizes the backend services.

Table 7: Backend Service Ecosystem and Technical Orchestration

Component	Technology	Utility & Responsibility
Gateway Layer	Spring Cloud Gateway 3.1.9	Dynamic routing, JWT validation, IP rate limiting, WebSocket proxy
Business Logic	Spring Boot 3.x + MyBatis-Plus	Service orchestration, type-safe CRUD, logical deletion (@TableLogic)
Security Framework	Spring Security 6.x + JWT	Stateless authentication, RBAC with @PreAuthorize, BCrypt password hashing
Caching Layer	Spring Data Redis	Session caching, rate limit counters, metadata cache
AI Integration (Tool)	DeepSeek API (HTTPS)	Intent routing (MEETING/SCHEDULE/ROUTE), parameter extraction, route translation
Inference Engine	Python FastAPI (port 8090)	LLM-based SQL generation from natural language questions via /infer endpoint
RAG Agent	Spring Boot 3.x (port 8085)	Document Q&A, vector retrieval, source citation, access workflow
Embedding Worker	Python FastAPI (rag-worker)	bge-m3 embedding generation, Markdown chunking
Document Parsing	Apache Tika	Multi-format document parsing for RAG ingestion pipeline
Vector Retrieval	Milvus Java SDK 2.3+	Semantic ANN search for RAG document retrieval
Object Storage	MinIO	Document file storage (PDF/DOCX/PPT), Milvus backend
Email Service	Resend API	Email verification code delivery, three-tier rate limiting
Map Services	Amap API (HTTPS)	Geocoding (multi-level fallback), direction calculation, polyline coordinate parsing

Data security is paramount in banking systems. Therefore, our platform's security design does not rely on a single line of defense, but rather incorporates defense in depth across all layers. Each layer is independently verified before a request can pass through, which make it significantly harder for a single component failure to lead to a data breach or security incident.

Figure 4 shows the complete user authentication flow in three stages, including the email verification code mechanism that supports both registration and code-based login.

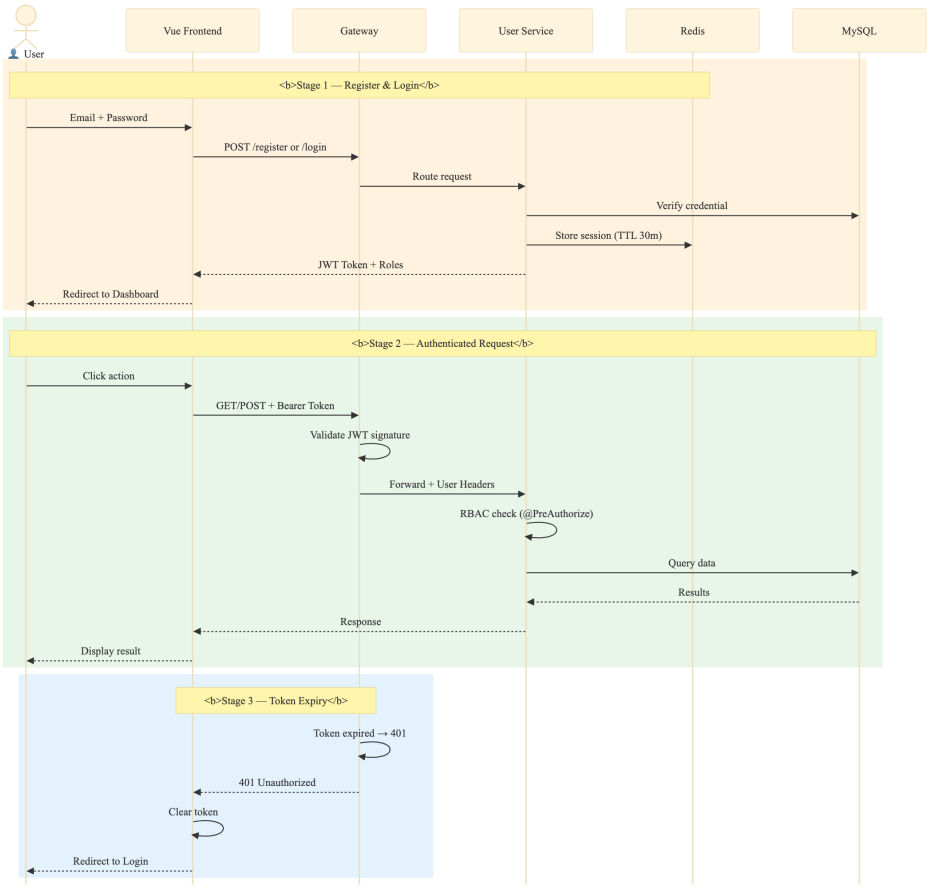

Figure 4: JWT Authentication and Email Verification Flow

The login process begins with an email verification code. After the user submits their email address on the front end, the request reaches the user service section. The service first checks the 60-seconds sending cooldown (Redis key rate:email:send: {email}), then queries the hourly and daily sending limits. After all rate limits are passed, it calls the Resend HTTP API using WebClient. The recipient email address is injected into the HTML template at the {email} placeholder, and the generated 6-digit random verification code is stored in Redis with a 5-minute TTL. The front-end receives a success response and the sample email is displayed in an Alert.

Specifically, the registration process requires the user to submit their email address, verification code, and password. The backend retrieves the verification code from Redis and compares it. If successful, it creates the user record (with the password hashed by BCrypt), immediately deletes the verification code from Redis, and returns a JWT. Attempts exceeding 5 incorrect verification attempts will cause the verification code to be forcibly invalidated and the user need to request a new one. For the multi-level rate limiting, the

code brute-force crack protection is covered in NFR3.

In subsequent requests, the frontend automatically appends a Bearer Token via an Axios interceptor. the gateway layer's JwtAuthGlobalFilter is responsible for signature verification, checking the validity period and extracting the payload. The whole process is statless—the backend does not keep session state—so the system can be scaled horizontally seamlessly. This approach avoids the common performance and availability issues associated with centralized session storage.

The CAPTCHA itself is also heavily reinforced: the 6-digit random number only lasts for 5 minutes and is immediately deleted from Redis after successful verification, ensuring single-use semantics; a maximum of 5 incorrect attempts trigger forced invalidation; the 60-seconds cooldown effectively prevents SMS bombing attacks, so the CAPTCHA security is not something we overlooked.

We adopted RBAC for our permission design, but only implemented a coarse-grained division of roles into three tiers—this "coarseness" was intentional. Different banks have vastly different organizational structures and reporting relationships; fine-tuning the details ourselves would inevitably not match a real deployment scenario. So, we chose to provide a clear foundational role framework and leave the detailed permission customization to the bank's own IT administrators.

Table 8: Pseudocode — listDocumentsForUser() Multi-Factor Document Access Rule

```
FUNCTION listDocumentsForUser(userId, userDeptId, userClearance)
    documents = queryAllDocuments()
    result = []
    FOR EACH doc IN documents:
        // Multi-factor access rule: a document is visible if ...
        IF doc.dept_id IS NULL THEN
            // Global document - accessible to all users
            doc.accessible = TRUE
        ELSE IF doc.dept_id = userDeptId
                 AND doc.security_level <= userClearance THEN
            // Same department with sufficient clearance
            doc.accessible = TRUE
        ELSE
            // Access denied - user may submit escalation request
            doc.accessible = FALSE
            doc.escalationType = "RAG_APPLY"
            // Manager can approve (Pending -> Approved) or deny (Pending -> Denied)
        END IF
        result.add(doc)
    END FOR
    RETURN result
END FUNCTION
```

The unified exit point for external communication is the REST API, all using HTTPS, with the TLS version locked at 1.3. WebSocket is forcibly upgraded to WSS in the production environment; this switch is handled in the code by checking the protocol in the Nginx X-Forwarded-Proto header, with the backend redirecting HTTP traffic to HTTPS.

4.5 Data Layer Architecture

For the data persistence layer, we adopted a multi-engine heterogeneous architecture. The main reason for not using a single database were the significant difference in storage requirements for different data types—storing vectors, relational data, and cache data all in MySQL would have created performance bottlenecks and architectural complexity. Instead, three specialized storage engines were deployed, each optimized for its specific data type.

All relational data resides on MySQL 8.0. We agreed on the database name agent_platform, character set utf8mb4, and collation utf8mb4_unicode_ci—while general_ci is more faster, it causes garbled characters in Chinese and emoji symbols. This design is essential for internationalization (including Traditional Chinese). The complete schema (9 tables) and seed data are documented in Appendix A.

The user and permission management uses five tables: sys_user stores account information, passwords are hashed using BCrypt, and it's linked to sys_department as foreign key, also recording security levels (1-3). sys_role defines three role codes (ROLE_ADMIN, ROLE_DEPT_ADMIN, ROLE_USER). sys_permission defines six granular permissions covering user management, role management, and department management. sys_user_role is a many-to-many association table connecting users to roles, and sys_role_permission connects roles to granular permissions. The primary identity management process relies on the RBAC model described in the security section. For document management, the sys_document table stores knowledge base content with clearance levels and department associations. Meeting room management uses meeting_room and meeting_schedule tables, where schedule records include room_id FK, booker, start_time, end_time, and attendee list.

During system initialization, we added a batch of seed data: four meeting rooms, five users with different roles and security levels, six departments, six permissions, a complete role allocation relationship tree, and eleven system documents covering banking compliance scenarios.

Cache and high-frequency operational data are handled by Redis 5.0+. Session tokens are stored in memory with configurable TTLs; atomic counting also rely on Redis when the gateway layer performs IP-level rate limiting (INCR + EXPIRE is the only strategy we use, it's simple, reliable, and effectively prevents race conditions). The JWT single-session mechanism is implemented through Redis keys token:username: {username}; on each login, the system invalidates any existing token for the same user.

Redis is also used for the tool's intelligent agent's schedule conflict detection. The key format is tool:schedule:{userId}. We store the meeting timestamps as scores using an ordered set. For overlapping queries, we use ZRANGEBYSCORE to retrieve entries whose startEpoch is within the query time window. The operation has logarithmic time complex-

ity for both insertion and query. For MySQL-Redis consistency, we adopted a scheme of "write MySQL first, then write Redis, and delete the Redis key on failure," this is basically a simple cache-aside pattern, prioritizing strong consistency for persistent data.

Semantic retrieval for RAG agents requires vector support, so we separately introduced Milvus 2.3+. During the selection process, we considered several vector engines, and Milvus, with its relatively advanced four-tier architecture (access layer, coordinator, worker node, and object storage), ultimately stood out for enterprise deployment scenarios. Milvus's collection schema is dynamically configured at startup, with dimension matching the embedding model dimension (default 4096), metric type set to IP (inner product), and index type supporting IVF_FLAT and HNSW.

Going forward, we plan to add a security post-filter to the RAG retrieval pipeline. The raw vector results returned by Milvus will not be directly passed through; instead, they will be secondarily filtered against the allowed document list computed during the pre-retrieval phase, guaranteeing enforcement even if the filter expression in the Milvus query itself fails.

Table 9 summarizes the data storage strategy.

Table 9: Multi-Modal Data Storage Strategy and Layered Access Patterns

Category	Engine	Primary Data Types	Access Pattern
Operational Layer	MySQL 8.0	Users, roles, departments, documents, meeting rooms, schedules, notifications	Transactional, ACID-compliant read/write
Caching Layer	Redis 5.0+	Session tokens, rate limit counters, metadata cache (hash), schedule conflicts (sorted sets)	High-concurrency, sub-millisecond reads
Object Storage	MinIO	PDF, DOCX, PPT files; Milvus backend objects	Document upload/download, RAG source document storage
Vector Layer	Milvus 2.3+	Document chunk embeddings (bge-small-zh), source metadata	Semantic approximate nearest neighbor search

4.6 Technology Stack Rationale

After discussion, the technology stack of our system was determined based on four main

principles: First, it should be able to be deployed locally to reduce reliance on external cloud services (because banking regulations require data to be stored on local servers); second, all components should work well with container technology (because we wanted the developer environment and the production environment to be as similar as possible); third, the technology stack should match the team's current skill set (backend students are more familiar with Java than Python, so AI services had a separate Python module for LLM inference tasks); fourth, we preferred ecosystems that have long-term support and active communities (so we can find an answer on Stack Overflow when hitting a problem).

Specifically, the system's backend framework uses Java + Spring Boot. This choice was primarily based on its mature enterprise-level ecosystem. A strong typing system effectively reduces implicit errors caused by type mismatches; Spring Security provides fine-grained method-level permission control support for RBAC. Each microservice is configured with an independent spring-boot-starter-parent, managed hierarchically through parent POMs to unify dependency versions.

Our gateway is Spring Cloud Gateway, rather than a traditional Servlet solution. (One issue with Servlets is that containers default to a blocking "one thread, one request" model. As the gateway acts as the hub for all service traffic, the extra thread overhead would have been substantial under high load. The reactive model is more suitable for the gateway's role as a proxy and filter, given its high-concurrency scenario. The gateway performs identity verification through a custom JwtAuthGlobalFilter, repackaging the token payload (user ID, roles, and permissions) into the request headers, unifing the previously scattered authentication logic.

The AI inference service ultimately chose Python + FastAPI, deployed on port 8090. Python's dominant position in the machine learning toolchain is unlikely to be replaced in the short term (compared to alternatives like Scala or R, its library ecosystem is dramatically richer). FastAPI offers performance close to Node.js while automatically generating Swagger documentation, which reduces the cost of inter-team interface coordination.

We settled on Vue 3 with TypeScript after weighing several options. While the team's front-end skill levels were uneven, Vue 3's learning curve proved less steep than alternatives, which mattered give our project schedule. TypeScript was not optional—it stopped numerous type errors at compile time that JavaScript would have silently let through. The

Composition API gave us logic reuse that we actually used across all the pages that call agents that share the same skeleton of request → progress → result display logic, while the Options API would have forced code duplication.

Containerization via Docker and Docker Compose was our choice for environment parity. MySQL, Redis, and Milvus all run as containers, which eliminates the “it works on my machine” problem. The Java services are each packaged as thin JARs, with dependencies separated from the application code to reduce image rebuild time during iteration.

5 Core Agent Implementation

This chapter covers the three agent modules that form the platform's intelligence layer—tool agent, code agent, and RAG agent. Each section discusses internal architecture, key algorithms, and data flow design. Pseudocode tables and sequence diagrams are included to illustrate the operational logic of each agent.

5.1 Tool Agent Implementation

This agent runs within a separate tool-agent service (Spring Boot, port 8083), providing three capabilities: meeting room management, schedule conflict detection, and route planning. Technically, business logic is divided into REST endpoints and specialized service classes; the AI intent module provides a unified entry point for natural language input.

5.1.1 Architecture and AI Intent Routing

The tool agent employs a dual-path design at the interaction layer: one side retains programmatic calls, while the other open a natural language entry point.

Programmatic calls use a separate REST endpoint, where the frontend or third-party system directly passes type parameters, making the logic straightforward. Natural language interactions are uniformly sent to the /tool/execute endpoint. The DeepSeek LLM (Starfire API, accessed via the iFlytek MaaS platform) identifies the user's intent and extracts parameters from the natural language message. The request is then routed to the corresponding handler.

We separate the natural language entry point from the programmatic entry point, rather than using AI routing for everything, primarily for cost considerations. DeepSeek's token consumption and response latency are much higher than a direct REST call (roughly 500–800ms versus 5–10ms for a local call). In cases where the frontend already knows exactly what operation is to be performed (button clicks), calling the AI again would be both unnecessarily slow and a waste of tokens—adding extra cost for no benefit.

We ultimately adopted the iFlytek MaaS (Model as a Service) platform, which provide an interface compatible with the OpenAI Chat Completions format. Therefore, the underlying HTTP client can directly use the standard OpenAI library call pattern without modifying the request body structure.

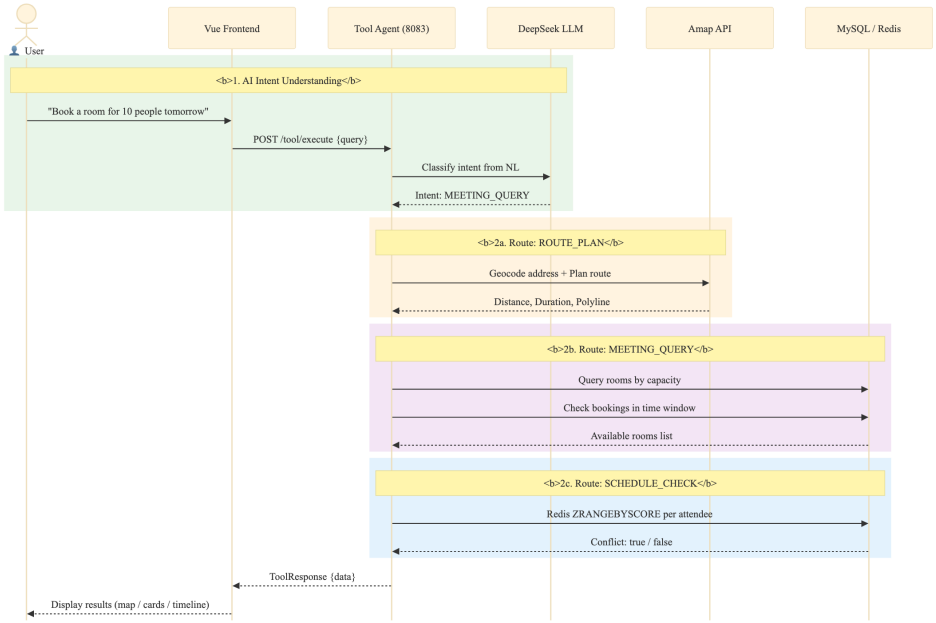

Figure 5: Tool Agent Task Sequence — DeepSeek AI Intent Routing to Specialized Handlers

Tables 10 and 11 show the AI intent routing pipeline where DeepSeek classifies user input into three intent types and dispatches to the corresponding handler, with a mock fallback on API failure.

Table 10: Pseudocode — handleAiIntent() AI Intent Classification and Routing

```
FUNCTION handleAiIntent(userInput: String): HandlerResult
    systemPrompt = PromptBuilder.create()
        .defineIntent("ROUTE_PLAN",
            params = ["from", "to", "mode"],
            description = "Route planning request")
        .defineIntent("MEETING_QUERY",
            params = ["date", "capacity", "equipment"],
            description = "Meeting room search")
        .defineIntent("SCHEDULE_CHECK",
            params = ["timeRange", "attendees"],
            description = "Schedule conflict detection")
        .build()
    response = deepSeekService.chat(
        system  = systemPrompt,
        user    = userInput,
        format  = "json_object",
        temp    = 0.3,
        maxTok  = 2048
    )
    intent = response.intentType
    params = response.parameters
    SWITCH intent:
        CASE "ROUTE_PLAN":
            RETURN routeHandler.handle(params.from, params.to,
               params.mode)
        CASE "MEETING_QUERY":
            RETURN meetingHandler.handle(params.date,
                       params.capacity, params.equipment)
        CASE "SCHEDULE_CHECK":
            RETURN scheduleHandler.handle(params.timeRange,
                       params.attendees)
        DEFAULT:
            THROW UnknownIntentException(intent)
    END SWITCH
END FUNCTION
```

Table 11: Pseudocode — DeepSeekService.chat() LLM API Call with Fallback

```
FUNCTION chat(system, user, format, temperature, maxTokens):
    DeepSeekResponse
    requestBody = {
        model:              config.deepseek.model,
        messages: [
            {role: "system", content: system},
            {role: "user",   content: user}
        ],
        response_format:    {type: format},
        temperature:        temperature,
        max_tokens:         maxTokens
    }
    TRY
        httpResponse = httpClient.send(
            method:  POST,
            url:     config.deepseek.baseUrl + "/chat/
                completions",
            headers: {
                "Authorization": "Bearer " + config.deepseek.
                    apiKey,
                "Content-Type":  "application/json"
            },
            body:    toJSON(requestBody),
            timeout: 60s
        )
        rawJson = httpResponse.choices[0].message.content
        parsed  = objectMapper.readValue(rawJson,
            DeepSeekResponse)
        RETURN parsed   // {intentType, parameters}
    CATCH Exception e:
        log.error("DeepSeek API call failed", e)
        RETURN fallbackMock()   // demo data, source = "mock"
    END TRY
END FUNCTION
```

The DeepSeek API configuration in application.yml uses environment variables with development defaults: AI_DEEPSEEK_API_KEY, AI_DEEPSEEK_BASE_URL (defaulting to https://maas-api.cn-huabei-1.xf-yun.com/v1), AI_DEEPSEEK_MODEL (defaulting to deepseek-v3), AI_DEEPSEEK_TIMEOUT (180s), and AI_DEEPSEEK_MAX_TOKENS (2000).

5.1.2 Meeting Room Management

For meeting room management, due to varying user permissions, we designed two sets of

interfaces base on user permissions and functions:

The first set is for regular users. Regular users has a simple request: “Check if there are any available rooms for this time slot.” Therefore, our ToolController only includes a single GET request: /tool/meeting-rooms. Parameters include startTime, endTime, and capacity. The backend performs a three-phase availability check, as shown in the pseudocode in Table 12.

The second set, the management side, uses a completely different logic (although the system administrator and department administrators maintain consistency). Because the Admin class needs to maintain meeting room resources, we provide a complete set of CRUD interfaces through MeetingRoomAdminController. In terms of security, it verifies the X-User-Roles header—this is simple but effective, because the header is injected by the gateway layer, not the client, and therefore cannot be forged.

Table 12: Pseudocode — queryRoomsWithStatus() Three-Phase Room Availability Algorithm

```
FUNCTION queryRoomsWithStatus(startTime, endTime, minCapacity):
    List<Map>
    // === Phase 1: Query all active meeting rooms ===
    query = new LambdaQueryWrapper<MeetingRoom>()
        .eq(MeetingRoom::status, 1)
    IF minCapacity <> NULL THEN
        query.ge(MeetingRoom::capacity, minCapacity)
    END IF
    rooms = meetingRoomMapper.selectList(query)

    // === Phase 2: Find booked room IDs in time range ===
    overlapping = bookingMapper.selectList(
        new LambdaQueryWrapper<MeetingSchedule>()
            .lt(MeetingSchedule::startTime, endTime)
            .gt(MeetingSchedule::endTime,   startTime)
    )
    bookedRoomIds = {b.roomId | b IN overlapping}

    // === Phase 3: Assemble availability response ===
    result = []
    FOR EACH room IN rooms:
        available = room.id NOT IN bookedRoomIds
        equipment = split(room.facilities, ",")
        result.add({
            id:          toString(room.id),
            name:        room.roomName,
            capacity:    room.capacity,
            location:    room.building + ", Floor " + room.
               floor,
            equipment:   equipment,
            available:   available,
            statusText:  IF available THEN "Available" ELSE "
               Booked",
            nextBooking: findNextBooking(room.id, startTime)
        })
    END FOR
    RETURN result
END FUNCTION
```

Table 13: Pseudocode — bookRoom() Atomic Conflict Check-and-Insert

```
FUNCTION bookRoom(roomId, startTime, endTime, booker, title):
   Long
    // Step 1 -- Atomic conflict check
    conflicts = scheduleMapper.selectCount(
        new LambdaQueryWrapper<MeetingSchedule>()
            .eq(MeetingSchedule::roomId,    roomId)
            .lt(MeetingSchedule::startTime, endTime)
            .gt(MeetingSchedule::endTime,   startTime)
    )
    IF conflicts > 0 THEN
        RETURN FALSE   // room already booked in this window
    END IF
    // Step 2 -- Insert new booking
    schedule = new MeetingSchedule()
    schedule.roomId    = roomId
    schedule.startTime = startTime
    schedule.endTime   = endTime
    schedule.booker    = booker
    schedule.title     = title
    schedule.status    = 1
    scheduleMapper.insert(schedule)
    RETURN schedule.id
END FUNCTION

FUNCTION getSchedulesForUsers(userIds, startTime, endTime):
   List<Schedule>
    RETURN scheduleMapper.selectList(
        new LambdaQueryWrapper<MeetingSchedule>()
            .in(MeetingSchedule::booker, userIds)
            .ge(MeetingSchedule::startTime, startTime)
            .le(MeetingSchedule::endTime,   endTime)
    )
END FUNCTION
```

The MeetingRoom entity (@TableName(“meeting_ room”)) maps to the database table with fields: id (Long, @TableId auto-increment), roomName (String), building (String, e.g., “Building A”), floor (String, e.g., “3F”), capacity (Integer), hasProjector/hasWhiteboard/hasVideoConf (Boolean), status (Integer, 0=available, 1=maintenance), and createTime (LocalDateTime).

Table 14 lists the Tool Agent REST API endpoints implemented in ToolController.

Table 14: Tool Agent REST API Endpoints

Method	Endpoint	Key Parameters	Description
POST	/tool/execute	toolType, naturalLanguage, parameters	Unified AI execution — DeepSeek LLM classifies intent and routes to handler
GET	/tool/meeting-rooms	startTime, endTime, capacity	Query rooms with time-based availability status and capacity filter
POST	/tool/meeting-room/book	roomId, booker, startTime, endTime, topic	Atomic check-then-insert room booking with conflict verification
POST	/tool/check-conflict	attendees[], startTime, endTime	Check schedule conflicts by attendee list via Redis ZRANGEBYSCORE
POST	/tool/schedule/create	userId, eventName, startTime, endTime	Create personal schedule (dual-write: MySQL room_id=0 + Redis sorted set)
POST	/tool/schedule/add	userId, eventId, startTime, endTime	Add schedule entry to Redis sorted set only
POST	/tool/schedule/remove	userId, eventId	Remove schedule entry from Redis sorted set by eventId prefix match
GET	/tool/schedules	date, users[]	Query schedules for specified users on a given date from MySQL
GET	/tool/my-schedules	X-User-Name header	Get authenticated user's bookings and personal schedules with room details
DELETE	/tool/my-schedule/{id}	X-User-Name header, path id	Cancel schedule: set status=0 in MySQL, remove from Redis if personal
GET	/tool/route	from, to, mode (default driving)	Plan route via Amap API with multi-level geocoding fallback and map data

5.1.3 Schedule Conflict Detection

Schedule conflict detection uses a dual-layer architecture: MySQL is the persistent store for all schedule data, while Redis serves as a fast query cache for temporal overlap detection. The ScheduleService handles CRUD operations on meeting_schedule records. Each schedule has a booker (the user who created it), optional room_id (0 for personal schedules that don't need a room), start_time/end_time, and an attendees field stored as a comma-separated list. Each user's schedules are indexed in Redis as a sorted set, where scores represent epoch timestamps and members encode event identifiers plus time ranges—this

structure allows O(log N) insertion and O(log N + M) range queries, where N is the number of schedules for that user and M is the number of results in the time window.

Figure 6 shows the schedule conflict detection algorithm.

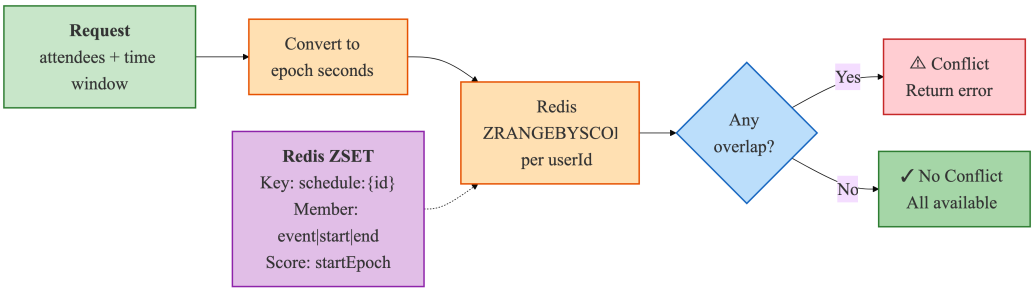

Figure 6: Schedule Conflict Detection — Redis Sorted Set-Based Temporal Overlap Algorithm

Each user's schedules are stored in a Redis sorted set at key tool:schedule:{userId}. Schedule entries are encoded as members with the format "{eventId}|{startEpoch}|{endEpoch}", where startEpoch and endEpoch are Unix timestamps. For conflict detection queries, ZRANGEBYSCORE retrieves all schedule entries whose startEpoch is within the specified query window. For each retrieved schedule, the algorithm checks if the stored endEpoch overlaps with the query range to filter out partial overlaps.

Tables 15, 16, and 17 show the schedule management subsystem including Redis sorted-set operations for temporal conflict detection, attendee-based conflict checking, and the MySQL-Redis dual-write consistency protocol.

Table 15: Pseudocode — Redis Sorted Set Schedule Operations (add / remove / checkConflicts)

```
FUNCTION addSchedule(userId, eventId, startTime, endTime)
    key      = "tool:schedule:" + userId
    startTs  = toEpochMillis(startTime)
    endTs    = toEpochMillis(endTime)
    member   = eventId + ":" + startTs + ":" + endTs
    redisTemplate.opsForZSet().add(key, member, startTs)
END FUNCTION

FUNCTION removeSchedule(userId, eventId)
    key     = "tool:schedule:" + userId
    members = redisTemplate.opsForZSet().range(key, 0, -1)
    FOR EACH member IN members:
        IF member.startsWith(eventId + ":") THEN
            redisTemplate.opsForZSet().remove(key, member)
            BREAK
        END IF
    END FOR
END FUNCTION

FUNCTION checkConflicts(userId, startTime, endTime): List<
   String>
    key     = "tool:schedule:" + userId
    startTs = toEpochMillis(startTime)
    endTs   = toEpochMillis(endTime)
    // Query schedules whose start time falls within window
    overlapping = redisTemplate.opsForZSet()
        .rangeByScore(key, startTs, endTs)
    conflicts = []
    FOR EACH member IN overlapping:
        [evtId, memStartTs, memEndTs] = split(member, ":")
        IF parseLong(memStartTs) < endTs THEN
            conflicts.add(evtId)
        END IF
    END FOR
    RETURN conflicts
END FUNCTION
```

Table 16: Pseudocode — POST /tool/check-conflict Attendee-Based Conflict Workflow

```
ENDPOINT POST /tool/check-conflict
  Request:  {attendees: String[], startTime: String, endTime:
     String}
  Response: {hasConflict: Boolean, conflictUser: String?}

FUNCTION checkConflict(attendees, startTime, endTime):
   ConflictResult
    FOR EACH userId IN attendees:
        conflicts = scheduleService.checkConflicts(userId,
                        startTime, endTime)
        IF conflicts IS NOT EMPTY THEN
            // Early exit -- return first conflict found
            RETURN {
                hasConflict:  TRUE,
                conflictUser: userId,
                conflicts:    conflicts
            }
        END IF
    END FOR
    RETURN {hasConflict: FALSE, conflictUser: NULL}
END FUNCTION
```

Table 17: Pseudocode — Schedule Dual-Write Pattern (MySQL + Redis Consistency)

```
// === Schedule Creation (dual-write) ===
ENDPOINT POST /tool/schedule/create
FUNCTION createSchedule(request): ScheduleResponse
    // Write 1 -- Persist to MySQL
    schedule = meetingRoomService.addPersonalSchedule({
        roomId:    0,       // personal schedule convention
        booker:    request.booker,
        startTime: request.startTime,
        endTime:   request.endTime,
        title:     request.title,
        status:    1
    })
    // Write 2 -- Cache to Redis for fast conflict queries
    eventId = "event_" + schedule.id
    scheduleService.addSchedule(request.booker, eventId,
                                request.startTime, request.
                                   endTime)
    RETURN {scheduleId: schedule.id, eventId: eventId}
END FUNCTION

// === Schedule Cancellation (dual-delete) ===
ENDPOINT POST /tool/schedule/cancel
FUNCTION cancelSchedule(scheduleId): void
    // Step 1 -- Soft delete from MySQL (preserves audit trail)
    scheduleMapper.update(null,
        new LambdaUpdateWrapper<MeetingSchedule>()
            .eq(MeetingSchedule::id, scheduleId)
            .set(MeetingSchedule::status, 0)
            .set(MeetingSchedule::updateTime, now())
    )
    // Step 2 -- Remove from Redis cache
    scheduleService.removeSchedule(currentUser, "event_" +
       scheduleId)
END FUNCTION
```

5.1.4 Route Planning

The route planning feature integrates the Amap Open API through the AmapService class, which provides geocoding, route calculation, and map data for visualization. The GET /tool/route endpoint accepts origin, destination, and optional waypoint parameters, returning a route response containing path coordinates, turn-by-turn instructions, total distance, and estimated duration. geocoding (converting addresses to coordinates) is implemented through Amap's geocoding API, with detailed request parameters and LLM-based route

description translation.

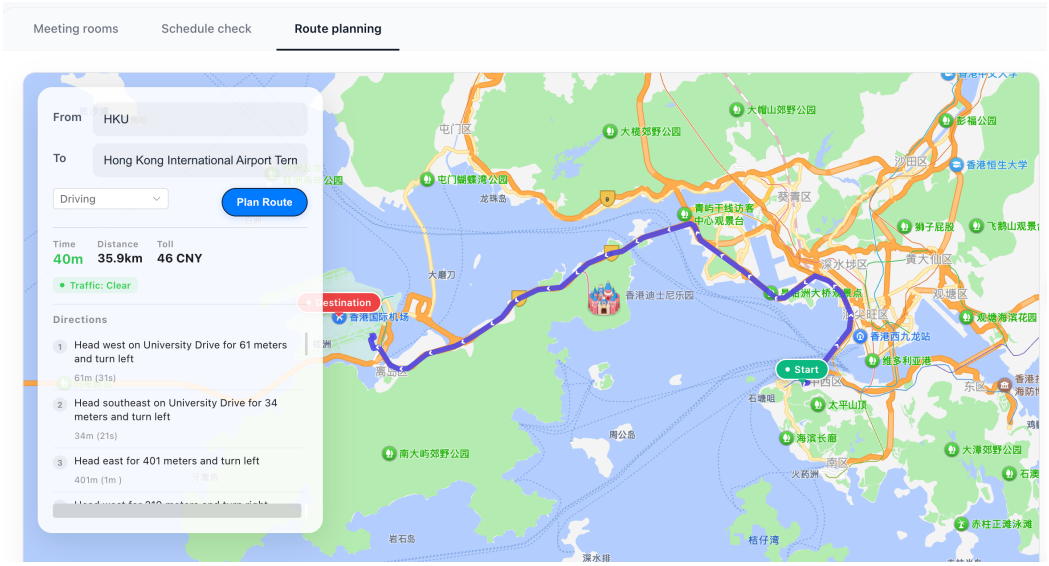

Figure 7: Route Planning Map Visualization — Interactive Map with Route Polyline and Navigation Steps

Table 18: Route Planning Three-Stage Geocoding and Navigation Pipeline

Stage	Method	API Endpoint	Input	Output	Fallback
1 – Geocoding	geocodeWithFallback()	`GETrestapi.amap.com/v3/geocode/geo`	Address string	Location (lng,lat)	POI text search, prepend “Beijing” + retry
2 – Route Calc	planDrivingRoute()	`GETrestapi.amap.com/v3/direction/driving`	Origin + Dest coords, strategy (0–4)	Distance, duration, polyline steps	N/A — returns error to caller
3 – Enrichment	parseRouteResult()	— (local processing)	Amap JSON response	Formatted text + coordinate arrays	N/A — keeps raw values on parse error

Table 19 shows the route geocoding pipeline with a three-level fallback strategy and LLM-based translation of navigation instructions.

Table 19: Pseudocode — geocodeWithFallback() Multi-Level Address Resolution

```
FUNCTION geocodeWithFallback(address: String): Location
    TRY
        response = httpGet(restapi.amap.com/v3/geocode/geo,
            {key: apiKey, address: URL_ENCODE(address)})
        IF response.geocodes IS NOT EMPTY THEN
            RETURN parseLocation(response.geocodes[0].location)
        END IF
    CATCH:  // fall through
    TRY
        // Fallback 1 -- POI text search
        response = httpGet(restapi.amap.com/v3/place/text,
            {key: apiKey, keywords: URL_ENCODE(address), offset
               : 1})
        IF response.pois IS NOT EMPTY THEN
            RETURN parseLocation(response.pois[0].location)
        END IF
    CATCH:  // fall through
    // Fallback 2 -- Prepend "Beijing" and retry geocoding
    RETURN geocodeWithFallback("Beijing " + address)
END FUNCTION

FUNCTION translateRouteToEnglish(routeData): RouteData
    fields = extractChineseFields(routeData)
    TRY
        translated = deepSeekService.chat(
            system = "Translate to English, keep JSON structure
               ",
            user   = toJSON(fields),
            format = "json_object")
        RETURN replaceUnits(translated)
            // e.g., "km" instead of Chinese units
    CATCH:
        RETURN routeData   // keep original Chinese
    END TRY
END FUNCTION
```

On the frontend, the MapContainer.vue component loads the Amap JS API v2.0 SDK through script injection with the API key and security code. The component creates a map instance with zoom level 12, centered on the starting coordinate. An indigo polyline renders the route path. Green and red custom marker icons indicate the start and end points, respectively. Turning instructions are displayed in a scrollable list below the map, with each instruction showing the road name, distance, and a directional icon.

5.2 Code Agent Implementation

The Code Agent lets users generate executable SQL queries from natural language descriptions, connecting business users' domain expertise with the SQL knowledge needed for direct database interaction.

5.2.1 Architecture Overview

Table 20 shows the four-stage pipeline of Code Agent.

Table 20: Code Agent Four-Stage Natural Language to SQL Pipeline

Stage	Service Class	Key Operation	Security Mechanism
1 – Metadata Extraction	MetadataCacheService	Query information_schema for table/column definitions; cache as JSON in Redis (key: code_agent:metadata:tables)	Provides whitelists (table names, column names) to the validation layer
2 – SQL Generation	OnnxCodeGenerationService	HTTP POST {question} → Python LLM inference server (port 8090); receives {sql, method}	N/A — generation only; output is untrusted at this stage
3 – SQL Validation	SqlValidationService	Five-layer defense-in-depth pipeline: (1) operation type, (2) forbidden keywords, (3) table whitelist, (4) column whitelist, (5) complexity limits	Blocks: non-SELECT, DDL/DML keywords, unrecognized tables/columns, overly complex queries
4 – SQL Execution	SqlExecutionService	Re-validate (defense in depth) → execute via JdbcTemplate with PreparedStatement	Parameterized queries prevent injection; read-only connection enforced

The CodeAgentController exposes six endpoints: POST /code/query (one-shot generate and execute, returns FullResult with SQL, columns, rows, elapsedMs), POST /code/generate (preview only, generates SQL without execution), POST /code/execute (execute only, runs previously generated SQL), GET /code/metadata (returns cached schema metadata), POST /code/metadata/refresh (manually triggers schema refresh), and GET /code/health (returns service status and LLM connectivity).

Figure 8 shows the Code Agent use case diagram. Business analysts interact with the system by submitting natural language queries through the three-step frontend interface.

The system processes requests by encoding database schema information, sending structured prompts to the Python LLM inference server, receiving generated SQL, running it through the five-layer validation pipeline, executing only validated queries via JdbcTemplate against MySQL, and returning results with the generated SQL and execution metadata.

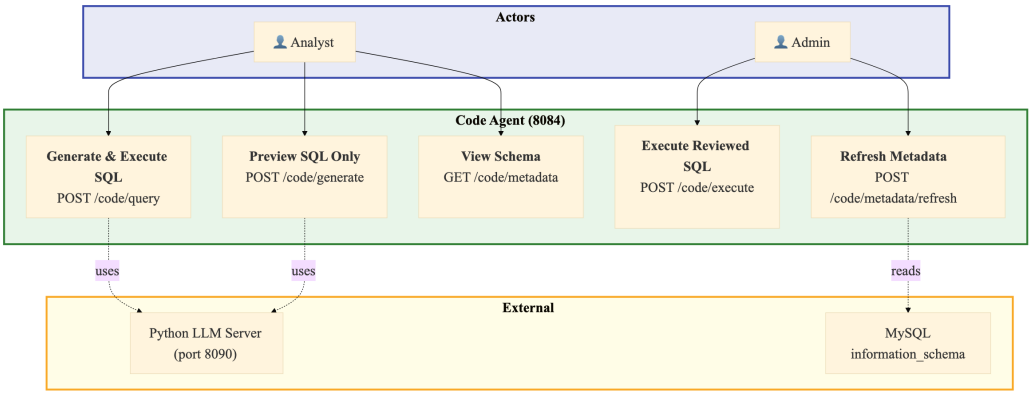

Figure 8: Code Agent Use Case Diagram — Natural Language to Safe SQL Execution

The use case covers schema-aware query generation, five-layer security validation, JdbcTemplate-based execution, and result presentation with export capabilities.

5.2.2 Python LLM Inference Server Integration

Table 21 shows the SQL generation HTTP proxy that forwards natural language questions to the Python LLM inference server and parses the returned SQL.

Table 21: Pseudocode — OnnxCodeGenerationService HTTP Proxy to Python Inference Server

```
// @Service, @ConditionalOnProperty(code-agent.onnx.enabled=
   true)
// Acts as HTTP proxy (NOT direct ONNX Runtime inference)
FUNCTION generateSQL(question: String): CodeGenerationResponse
    payload = {question: question}
    TRY
        httpResponse = httpClient.send(
            method:  POST,
            url:     config.serverUrl,  // default: http://
               localhost:8090/infer
            headers: {"Content-Type": "application/json"},
            body:    toJSON(payload),
            connectTimeout: 10s,
            responseTimeout: 60s
        )
        responseBody = objectMapper.readTree(httpResponse.body)
        sql    = responseBody.get("sql").asText()
        method = responseBody.get("method").asText()
        IF sql IS BLANK THEN
            RETURN failure("Generated SQL is empty")
        END IF
        RETURN success(sql, method)
    CATCH Exception e:
        log.error("SQL generation failed", e)
        RETURN failure(e.message)
    END TRY
END FUNCTION

// Design note: Any LLM serving framework can back port 8090
// as long as it accepts {question} and returns {sql, method}.
// Reference: Python inference server at data/infer_server.py
// hosts fine-tuned T5 model (data/fine-tuned-t5-banking/).
```

5.2.3 Five-Layer SQL Whitelist Validation

The SqlValidationService uses a defense-in-depth validation pipeline with five sequential layers, each of which can reject SQL that violates its security rule. The validation is configurable through CodeAgentProperties in application.yml.

Figure 9 shows the pipeline from natural language input to safe SQL execution. The five validation layers run in sequence.

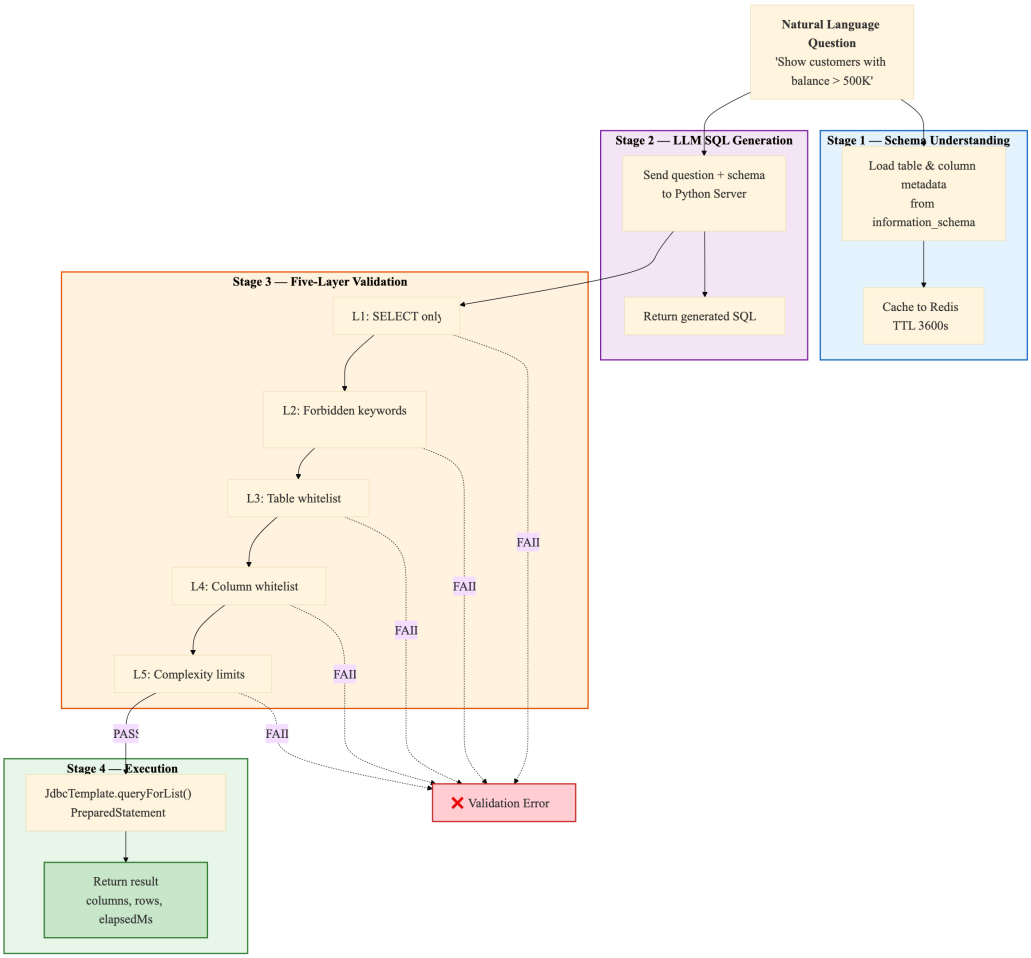

Figure 9: SQL Generation and Five-Layer Validation Pipeline

Tables 22, 23, 24, 25, and 26 show the five-layer SQL whitelist validation pipeline: operation type restriction, forbidden keyword blacklist, table name whitelist, column name whitelist, and query complexity limits.

Table 22: Pseudocode — validateOperation() SQL Operation Type Check (Layer 1)

```
FUNCTION validateOperation(sql: String): ValidationResult
    normalized = collapseWhitespace(sql).trim().uppercase()
    allowedOps = config.getAllowedOperations()  // default: ["
       SELECT"]
    FOR EACH op IN allowedOps:
        IF normalized.startsWith(op) THEN
            RETURN PASS
        END IF
    END FOR
    RETURN FAIL("Only SELECT operations are permitted")
END FUNCTION
```

Table 23: Pseudocode — validateForbiddenKeywords() Keyword Blacklist Scan (Layer 2)

```
FUNCTION validateForbiddenKeywords(sql: String):
   ValidationResult
    upperSQL = sql.uppercase()
    // Default blacklist: DROP, DELETE, INSERT, UPDATE, ALTER,
    //   TRUNCATE, CREATE, EXEC, EXECUTE, UNION, INTO,
    //   LOAD_FILE, OUTFILE, DUMPFILE
    FOR EACH keyword IN config.getForbiddenKeywords():
        pattern = "\\b" + keyword + "\\b"  // word-boundary
           regex
        IF pattern IS FOUND IN upperSQL THEN
            RETURN FAIL("Forbidden keyword detected: " +
               keyword)
        END IF
    END FOR
    RETURN PASS
END FUNCTION
```

Table 24: Pseudocode — validateTableNames() Table Whitelist Verification (Layer 3)

```
FUNCTION validateTableNames(sql: String): ValidationResult
    allowedTables = metadataCacheService.getAllowedTableNames()
    IF allowedTables IS EMPTY THEN
        log.warn("Metadata cache empty - skipping table
           whitelist")
        RETURN PASS   // skip when Redis/metadata unavailable
    END IF
    // Extract table references from FROM and JOIN clauses
    pattern = "\\b(?:FROM|JOIN)\\s+`?(\\w+)"  // case-
       insensitive
    extracted = findAllMatches(pattern, sql)
    FOR EACH tableName IN extracted:
        IF tableName NOT IN allowedTables THEN
            RETURN FAIL(
                "Table not allowed: " + tableName
                + ". Allowed: " + allowedTables)
        END IF
    END FOR
    RETURN PASS
END FUNCTION
```

Table 25: Pseudocode — validateColumnNames() Column Whitelist Verification (Layer 4)

```
FUNCTION validateColumnNames(sql: String): ValidationResult
    // Skip for SELECT * or SELECT COUNT(*) queries
    IF sql CONTAINS "SELECT *" OR sql CONTAINS "SELECT COUNT(*)
       " THEN
        RETURN PASS
    END IF
    allowedColumns = metadataCacheService.getAllowedColumns()
    IF allowedColumns IS EMPTY THEN
        log.warn("Metadata cache empty - skipping column
           whitelist")
        RETURN PASS   // skip when metadata unavailable
    END IF
    // Extract column spec between SELECT and FROM
    match = regex("SELECT\\s+(.+?)\\s+FROM", sql,
       CASE_INSENSITIVE)
    IF match NOT FOUND THEN
        RETURN PASS   // cannot parse, allow through
    END IF
    columnsPart = match.group(1)
    FOR EACH colSpec IN split(columnsPart, ","):
        colRef = extractColumnReference(colSpec)
        // Support both table.column and bare column
        IF colRef IS qualified (t.col) THEN
            IF colRef NOT IN allowedColumns[colRef.table] THEN
                RETURN FAIL("Column not allowed: " + colRef)
            END IF
        ELSE
            found = ANY table IN allowedColumns
                    WHERE colRef IN allowedColumns[table]
            IF NOT found THEN
                RETURN FAIL("Column not allowed: " + colRef)
            END IF
        END IF
    END FOR
    RETURN PASS
END FUNCTION
```

Table 26: Pseudocode — validateComplexity() Query Complexity Limits (Layer 5)

```
FUNCTION validateComplexity(sql: String): ValidationResult
    // Count table references
    tableCount = countMatches(
        "\\b(?:FROM|JOIN)\\s+`?(\\w+)", sql)
    maxTables  = config.getMaxTablesPerQuery()  // default: 3
    IF tableCount > maxTables THEN
        RETURN FAIL(
            "Too many tables: " + tableCount
            + " (max: " + maxTables + ")")
    END IF
    // Count conditions (AND/OR + 1 for base condition)
    conditionCount = 1
        + countMatches("\\bAND\\b", sql)
        + countMatches("\\bOR\\b",  sql)
    maxConditions = config.getMaxConditions()  // default: 10
    IF conditionCount > maxConditions THEN
        RETURN FAIL(
            "Too many conditions: " + conditionCount
            + " (max: " + maxConditions + ")")
    END IF
    RETURN PASS
END FUNCTION
```

Table 27 summarizes the five validation layers with their default configurations.

Table 27: SQL Whitelist Validation Rules (Five-Layer Defense)

Layer	Check	Default Configuration	Action on Violation
1. Operation	SQL must start with allowed operation	[“SELECT”] only	Block execution, return error with permitted operations
2. Keywords	Word-boundary regex blacklist scan	DROP, DELETE, INSERT, UPDATE, ALTER, TRUNCATE, CREATE, EXEC, EXECUTE, UNION, INTO, LOAD_ FILE, OUTFILE, DUMPFILE	Block execution, identify forbidden keyword
3. Table Names	FROM/JOIN tables vs. information_schema	All BASE TABLEs in agent_-platform database	Block execution, show allowed table set
4. Column Names	SELECT columns vs. table column definitions	Per referenced table from metadata cache; skip on empty cache	Block execution, identify invalid column
5. Complexity	Table count + condition count	Max 3 tables, max 10 conditions (AND/OR count + 1)	Block execution, show actual vs. max counts

5.2.4 Metadata Caching and Execution

Tables 28, 29, and 30 show the schema metadata cache loading from information_schema to Redis, the cache lifecycle management with scheduled refresh, and thedefense-in-depth SQL execution flow.

Table 28: Pseudocode — refreshAllMetadata() Schema Metadata Cache Loading

```
FUNCTION refreshAllMetadata(): Map<String, TableMetadata>
    conn = dataSource.getConnection()
    database = config.getDatabaseName()
    // Query 1 -- All BASE TABLE names
    tables = conn.query(
        "SELECT TABLE_NAME FROM information_schema.TABLES
         WHERE TABLE_SCHEMA = ? AND TABLE_TYPE = 'BASE TABLE'",
        database)
    metadataMap = {}
    FOR EACH tableName IN tables:
        // Query 2 -- Column metadata
        columns = conn.query(
            "SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE,
                    COLUMN_COMMENT, COLUMN_KEY
             FROM information_schema.COLUMNS
             WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?
             ORDER BY ORDINAL_POSITION",
            database, tableName)
        // Query 3 -- Table comment
        comment = conn.querySingle(
            "SELECT TABLE_COMMENT FROM information_schema.
               TABLES
             WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?",
            database, tableName)
        metadataMap[tableName] = {
            columns: columns,
            comment: comment
        }
    END FOR
    conn.close()
    // Serialize and cache to Redis
    json = objectMapper.writeValueAsString(metadataMap)
    redisTemplate.opsForHash().put(
        "code_agent:metadata", "tables", json)
    redisTemplate.expire(
        "code_agent:metadata", config.getCacheTtl())  //
           default: 3600s
    RETURN metadataMap
END FUNCTION
```

Table 29: Pseudocode — MetadataCacheManager Cache Lifecycle Management

```
// @Component - manages metadata cache lifecycle

// === Startup - initial cache load ===
@EventListener(ApplicationReadyEvent)
FUNCTION onApplicationReady():
    TRY
        metadataCacheService.refreshAllMetadata()
        log.info("Metadata cache loaded successfully")
    CATCH Exception e:
        log.warn("Database not ready. Manually refresh via:")
        log.warn("  POST /api/code/metadata/refresh")
    END TRY
END FUNCTION

// === Periodic refresh - every 30 minutes ===
@Scheduled(fixedRate = 1800000)
FUNCTION scheduledRefresh():
    TRY
        metadataCacheService.refreshAllMetadata()
    CATCH Exception e:
        log.error("Scheduled metadata refresh failed", e)
    END TRY
END FUNCTION

// === Fallback - direct query when Redis unavailable ===
FUNCTION getAllTableMetadata(): Map<String, TableMetadata>
    TRY
        cached = redisTemplate.opsForHash()
            .get("code_agent:metadata", "tables")
        IF cached NOT NULL THEN
            RETURN deserializeJSON(cached)
        END IF
    CATCH:  // Redis unavailable - fall through
    // Direct DB query as fallback
    RETURN refreshAllMetadata()
END FUNCTION
```

Table 30: Pseudocode — SqlExecutionService Defense-in-Depth SQL Execution

```
FUNCTION execute(sql: String): QueryResult
    // Defense in depth - re-validate even from trusted
       pipeline
    validation = sqlValidationService.validate(sql)
    IF validation IS NOT PASS THEN
        RETURN failure(validation.errorMessage)
    END IF
    // Execute via PreparedStatement (JdbcTemplate handles this
       )
    TRY
        rows = jdbcTemplate.queryForList(sql)
        RETURN success({
            columns:   extractColumnNames(rows),
            data:      rows,
            rowCount:  rows.size(),
            elapsedMs: measureElapsed()
        })
    CATCH DataAccessException e:
        RETURN failure("Query execution failed: " + e.message)
    END TRY
END FUNCTION

FUNCTION generateAndExecute(question, generationService):
   FullResult
    genResult = generationService.generateSQL(question)
    IF genResult IS FAILURE THEN
        RETURN FullResult.failure(genResult.error)
    END IF
    execResult = execute(genResult.sql)
    RETURN FullResult({
        success:         execResult.success,
        sql:             genResult.sql,
        inferenceMethod: genResult.method,
        columns:         execResult.columns,
        rows:            execResult.data,
        rowCount:        execResult.rowCount,
        elapsedMs:       execResult.elapsedMs
    })
END FUNCTION
```

The test dataset provides three core tables for evaluation and demonstration: bank_ customer (5 rows with customer_ no, name, id_ card, phone, risk_ level LOW/MEDIUM/HIGH, status), bank_account (5 rows with account_no, customer_no FK, account_type, balance, currency HKD/USD, open_date, status), and bank_transaction (5 rows with transaction_no, account_no FK, type DEPOSIT/WITHDRAW/TRANSFER,

amount, transaction_date, description). The test data is designed to cover common banking query patterns including customer summaries, balance aggregations, transaction histories, and multi-table joins.

5.3 RAG Agent Implementation

5.3.1 RAG-agent Microservice Architecture

The RAG agent microservice runs on port 8085 and is integrated into Spring Cloud Gateway via the new /api/rag/** route. The Gateway's JwtAuthGlobalFilter validates the JWT token and forwards the X-User-Id, X-Username, X-User-Dept-Id, and X-User-Security-Level headers to the rag-agent service, enabling permission-aware document retrieval.

5.3.2 Data Structure Design

The existing sys_document table remains the document master table, storing title, content, dept_id, and security_level. Three additional tables support the RAG pipeline. rag_document_chunk stores chunk-level records with document_id FK, chunk_index, content, content_hash, and char_count. rag_milvus_vector records the mapping between chunks and their vector IDs in Milvus, enabling chunk-level lifecycle management. rag_query_log records user queries, retrieved chunks, generated answers, latency metrics, and feedback for monitoring and evaluation.

5.3.3 Milvus Vector Database Integration

Milvus is used as the vector database. Docker Compose is extended with Milvus Standalone and its dependencies (etcd for metadata coordination, MinIO for object storage). On startup, rag-agent initializes a Milvus collection with dimension matching the embedding model (default 4096), IP metric type, and configurable index type (default HNSW). The Milvus collection schema is configured with fields: id (primary key, auto-generated), doc_id, chunk_id, dept_id, security_level, and embedding (FloatVector).

5.3.4 Document Chunking Strategy

Documents are processed with a structure-aware chunking strategy. Documents are first split by Markdown headings and paragraph boundaries to preserve document hierarchy.

When a paragraph exceeds the target chunk size (default 500 characters), it is split into overlapping chunks with a configurable overlap of 100 characters. Each chunk inherits the metadata of the source document: document ID, department ID, and security level.

5.3.5 Embedding Generation

Both document chunks and user questions need to be embedded into a shared vector space. The EmbeddingService interface defines embed(text) and embedBatch(texts) methods, and its implementation can support callable external API embedding or local ONNX embedding model. The module is designed to be replaceable—users can plug in different embedding models by changing a single service configuration.

5.3.6 Permission Security Filtering

Permission filtering ensures that the retrieved answers will never disclose the content of document beyond the user's authorized scope. The platform's applies the existing access control model at multiple levels. First, in the pre-retrieval phase, the system computes the set of document IDs the user is allowed to access based on their department membership and security clearance. Second, during vector retrieval, Milvus receives an expression filter restricting the search scope. Third, after retrieval, the results are re-verified against the user's permissions to prevent edge cases where Milvus filter expressions may not fully enforce the access policy.

5.3.7 Semantic Retrieval Flow

The retrieval flow first encodes the user's question with the same embedding model as the document indexing. Milvus performs an ANN (Approximate Nearest Neighbor) search within the allowedDocumentIds scope. The top-K most similar chunks are returned with their similarity scores and metadata. The similarity scores are computed as inner product (IP) between the query vector and document vectors.

5.3.8 LLM Answer Generation with Citations

The LLM (configured through RAG_LLM_API_KEY, RAG_LLM_BASE_URL, RAG_LLM_MODEL) is invoked by rag-agent to generate responses. The topK retrieved chunks are assembled into a structured prompt including the question, the context chunks with

source metadata, and instructions for generating a concise answer with citations. The LLM response includes answer text and a list of referenced sources. The frontend renders citations with clickable document references showing document title, chunk snippet, similarity score, and security level badge.

5.3.9 RAG API Endpoints

The RAG agent maps the RESTful API endpoints, and all endpoints are protected by Gateway-level JWT authentication. The POST /rag/query is the core Q&A endpoint, accepting {question, topK} and returning {answer, sources[], latency}. POST /rag/index/rebuild triggers a full re-indexing with a new Milvus collection. POST /rag/index/document/{id} indexes a single document with chunk-clean-rebuild logic. DELETE /rag/index/document/{id} deletes all chunks and vectors belonging to a document. GET /rag/health returns the agent's status, Milvus connectivity, and embedding model info.

5.3.10 Task Service Integration

The RAG capability has been integrated into the task-service, replacing the placeholder response ("RAG Agent is currently offline") with actual rag-agent calls. When a RAG-type task is submitted, the task-service will call the RAG agent's POST /rag/query endpoint and push the execution progress through WebSocket: 20% (checking permissions and computing document scope), 40% (performing vector retrieval), 70% (generating answer with LLM), and 100% (complete). The RAG response will be saved in task_record.output, enabling the Task Center and Copilot to use the same RAG capability through the existing orchestration framework.

5.3.11 Frontend RAG Workbench

The current /app/rag page includes: a question input area with loading states; an answer display section; a citation panel listing each referenced document with title, chunk snippet, similarity score, security level badge, and clickable document link; and a permission filter indicator showing the count of blocked documents with expandable details. The Copilot intent routing is updated to redirect RAG intents to /app/rag?query=xxx, in order that users can enter policy or document questions through the Copilot and arrive at the RAG Q&A

flow, and their questions have been pre-filled.

5.3.12 Document-RAG Index Synchronization

A synchronization mechanism ensures that the document library remains in sync with the RAG index. The initial version provides an admin-accessible "Rebuild Index" button on the RAG workbench. This button could be triggered via POST /rag/index/rebuild for manual full knowledge base reconstruction. The incremental synchronization includes: document creation triggers single-document indexing through POST /rag/index/document/{id}; document updates re-process only the affected document; document deletion removes all chunk records and corresponding vectors. The content hash comparison feature enables change detection: only re-processing the chunks whose content_hash differs from the stored value avoids unnecessary embedding costs.

5.3.13 Docker Deployment Configuration

Docker Compose is extended with new services: rag-agent (Spring Boot on port 8085), Milvus Standalone, etcd (Milvus metadata coordination), MinIO (Milvus object storage) as well as optionally rag-worker (local embedding model server). New environment variables include: `RAG_SERVICE_URI`, `MILVUS_HOST`, `MILVUS_PORT`, `RAG_EMBEDDING_PROVIDER`, `RAG_EMBEDDING_MODEL`, `RAG_EMBEDDING_DIM`, `RAG_LLM_API_KEY`, `RAG_LLM_BASE_URL`, `RAG_LLM_MODEL`, `RAG_MILVUS_COLLECTION`, `RAG_MILVUS_INDEX_TYPE`, and `RAG_TOP_K`. The services include dependency ordering so that Milvus and its dependencies start before rag-agent. The goal is a single `docker compose up --build -d` command that brings the complete platform online.

5.3.14 Technical Challenges and Testing Strategy

(1) Permission security: RAG retrieval must not leak document content through answers. Defense in depth with pre-retrieval filtering, in-query Milvus predicates, and post-retrieval re-verification prevents single-point filter failures. (2) MySQL-Milvus consistency: document updates, deletions, and re-indexing require coordinated cleanup of old chunks and vectors; content_hash-based change detection reduces unnecessary re-processing. (3) Embedding quality: retrieval accuracy depends on embedding model quality for Chinese financial texts. (4) LLM hallucination: citations and source verification mitigate factual

errors. (5) Performance: ANN search latency, embedding batch throughput, and LLM response time must meet service targets under concurrent load. Testing covers: admin access to high-clearance documents; standard users restricted to same-department and clearance-allowed documents; low-clearance users blocked from confidential documents; RAG_APPLY approval enabling temporary access; index refresh after document updates; graceful error handling when Milvus or LLM is unavailable; and frontend display with accessibility verification.

5.4 Copilot — A Unified Intelligent Dispatcher Above the Three Agents

The three agents described in Sections 5.1–5.3 each solve a specific domain problem, but from the user's perspective, they still require knowing *which* agent to use before typing a question. A bank employee asking "Which meeting rooms are free this afternoon?" should not need to understand that this goes to the Tool Agent rather than the Code Agent. The Copilot solves this by sitting above all three agents as a single natural language entry point: the user says what they need, and the Copilot figures out where it should go.

5.4.1 Design Rationale

The Copilot is not a fourth agent with its own domain capability. It is a **dispatcher** — its only job is to understand the user's intent, route the request to the right agent, and present the result back in a unified chat interface. This design follows a simple principle: the platform has exactly one text box the user needs to care about, and it works across every page.

Table 31 summarizes the Copilot's position relative to the three agents.

	Three Domain Agents	**Copilot**
Role	Execute domain-specific tasks (SQL, retrieval, tools)	Classify intent and dispatch
Input	Already-routed request with known type	Raw natural language, any topic
Output	Domain result (SQL rows, document excerpts, room lists)	Dispatch decision + agent result in chat
Scope	One agent = one capability	Cross-agent, cross-page
User interaction	Page-specific UI (tables, maps, panels)	Floating chat widget, always accessible

Table 31: Copilot vs. Three Domain Agents — Role Comparison

5.4.2 Architecture Overview

The Copilot spans three layers of the platform. On the frontend, a Vue 3 floating widget (CopilotWidget.vue) renders inside the global MainLayout, making it accessible from every page. On the backend, a Python Flask intent classifier (port 8090, shared with the Code Agent's inference server) performs the core dispatch logic. Between them, the Task

Service provides asynchronous execution and WebSocket-based progress streaming for long-running agent calls.

Figure 10 shows how a user query flows through the Copilot system.

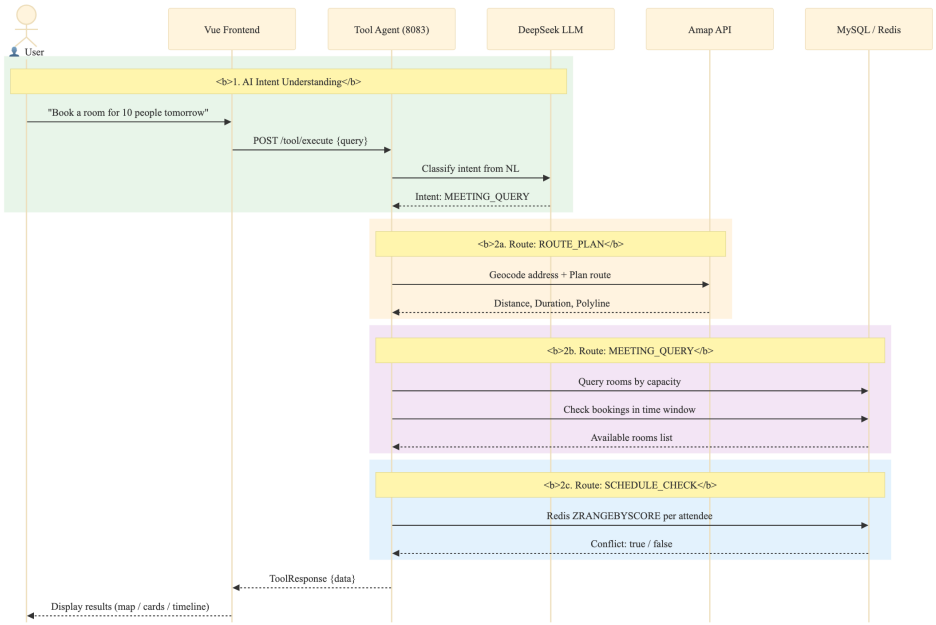

Figure 10: Copilot Query Flow — From Natural Language to Agent Execution

The flow has four stages. First, the user types a question into the Copilot widget. Second, the widget sends the question to `POST /api/dashboard/route`, which the Gateway rewrites to `/route` on the Python Flask server. Third, Flask runs a three-tier classification pipeline and returns an intent label. Fourth, the frontend dispatches based on the label: chat replies are shown inline, while domain tasks (TOOL, RAG) are submitted to the Task Service for asynchronous execution with progress streaming.

5.4.3 Three-Tier Intent Classification

The Python Flask `/route` endpoint is the Copilot's brain. It implements a three-tier classification strategy that balances cost, latency, and accuracy, shown in Table 32.

Tier	Mechanism	What It Catches	Cost
Tier 1	Pattern blocklist (40+ regex rules)	Jailbreak attempts, non-banking topics, prompt injection	Zero (no API call)
Tier 2	Keyword greeting match	"Hello", "What can you do", simple greeting phrases	Zero
Tier 3a	LLM classification (temperature=0.1, max_tokens=10)	CODE, TOOL, RAG, CHAT, CLARIFY, SETTINGS	∼10 tokens per query
Tier 3b	Keyword fallback (LLM unavailable)	Same six categories, lower accuracy	Zero

Table 32: Three-Tier Intent Classification Strategy

Tier 1 acts as a pre-screening safety net. Before any LLM call is made, the user's input is checked against a hardcoded list of approximately 40 blocked patterns. These cover jailbreak prompts (e.g., "ignore previous instructions"), role-play attempts ("pretend you are a hacker"), code injection, and entirely off-topic requests (jokes, recipes, gaming). Blocked queries are caught instantly and receive a polite refusal without consuming any API tokens.

Tier 2 catches simple greetings. If the user says "Hello" or "What can you do?" or the Chinese equivalent, the system responds with a friendly introduction listing the four capabilities (database queries, policy Q&A, meeting and schedule tools, route planning) — again with zero LLM cost.

Tier 3a is the core classification layer. The user's question is sent to the LLM with a system prompt defining six intent categories:

- **CODE**: Database queries, statistics, data lookups — anything that translates to SQL.
- **TOOL**: Meeting room booking, schedule conflict checking, route planning.
- **RAG**: Policy documents, regulations, concept definitions, compliance guidelines.
- **CLARIFY**: The user wants to book a room or plan a route, but has not provided enough parameters (e.g., no time specified for a meeting, no destination for a route).
- **SETTINGS**: Password changes, profile configuration.
- **CHAT**: Greetings, casual conversation, or anything outside the platform's scope.

The LLM is configured with temperature 0.1 and max_tokens 10 — the response is a single word, not a sentence. This keeps classification deterministic and cheap. For CLARIFY and CHAT intents, a second LLM call generates a conversational reply with appropriate follow-up questions or explanations.

Tier 3b activates only when the LLM API is unreachable. A pure keyword-matching fallback scans for terms associated with each intent — for CODE: terms related to databases, statistics, and data queries; for TOOL: terms related to meeting rooms, bookings, and schedules — and defaults to RAG when no keywords match. This ensures the Copilot degrades gracefully rather than failing completely.

5.4.4 Frontend Dispatch and User Experience

After the intent is classified, the frontend widget takes over with type-specific handling, summarized in Table 33.

Intent	Widget Action	User Sees
CHAT	Display reply text directly in chat bubble	The LLM's conversational response
CLARIFY	Save context, ask follow-up question	"Could you tell me what time you need the room?" — next user message prepends saved context
SETTINGS	Navigate to settings page	The settings form
CODE	Fire custom event, navigate to /app/code	The Code Agent page with the question pre-filled
TOOL	Submit task → WebSocket progress → result card	Animated processing bar with rotating status messages, then a result card (room list, conflict report, or route map)
RAG	Submit task → WebSocket progress → result card	A citation card with document titles, similarity scores, and security badges

Table 33: Intent Dispatch — Frontend Behavior by Classification Result

The Copilot widget itself is a floating component: a 52px circular icon button fixed to the bottom-right corner of every page. When clicked, it expands into a 440×620px chat panel with a morphing animation. This design means the user never leaves their current page to ask a question — the Copilot comes to them.

During agent execution, the input area is replaced by an animated processing bar. A puls-

ing blue dot, a water-ripple effect radiating from the input, and a set of status messages ("Analyzing your request...", "Identifying intent...", "Routing to the right agent...", "Processing with AI model...", "Gathering results...", "Almost there...") rotated every 1.5 seconds give the user continuous feedback. A live elapsed-time counter ticks every 100ms, so the user knows the system has not frozen.

When a task completes, the raw JSON output from the agent is transformed into a human-readable result card. Meeting queries show room names with colored availability dots, capacity badges, and location. Route plans show distance, duration, and traffic status. RAG answers show citation counts, blocked document counts, and a "View details" link. Conflict detections show either a green all-clear or an amber warning with the conflicting schedule details.

Session history is persisted in the browser's localStorage under the key prefix `copilot_session_`, with a session index limited to 20 entries. Users can review past conversations or start a fresh session from the history panel.

5.4.5 Multi-Turn Clarification

The CLARIFY intent enables a simple but effective multi-turn dialogue pattern. If a user types "Book a meeting room," the classifier recognizes that critical parameters are missing and returns CLARIFY. The widget saves the original query into a `sessionContextQuery` variable and displays the LLM's follow-up question (e.g., "What time and how many people?"). When the user replies "Three o'clock, five people," the widget prepends the saved context to form a combined query — "Book a meeting room [context: Three o'clock, five people]" — and re-classifies. This time, with all parameters present, the classifier returns TOOL and the request proceeds to the Tool Agent.

5.4.6 Cross-Page Communication

Because the Copilot widget is rendered once in MainLayout and persists across page navigation, it needs a mechanism to deliver agent results to whichever page the user is currently viewing. Two channels handle this:

1. **Browser Custom Events**: The widget dispatches `CustomEvent` objects on the `window` object — `copilot-tool-result` for Tool Agent results,

`copilot-code-query` for Code Agent queries. Target page components listen for these events and react accordingly.

2. **localStorage**: As a safety net for pages that mount *after* the event was dispatched (e.g., the widget navigated to a new page before the subscribing component existed), results are also written to localStorage keys. The target page checks these keys on mount and processes any pending result.

5.4.7 Second-Level Intent Classification in the Tool Agent

When the Copilot classifies a query as TOOL, the Task Service forwards it to the Tool Agent's `/tool/execute` endpoint with `toolType: "AI"`. At this point, a second classification happens inside the Tool Agent itself. The `handleAiIntent()` method sends the query to DeepSeek with a system prompt asking it to distinguish between three sub-intents: MEETING_QUERY, SCHEDULE_CHECK, and ROUTE_PLAN. The LLM returns a JSON object with the sub-intent and extracted parameters, which the Tool Agent uses to call the appropriate handler.

This two-level classification — Copilot first decides *which agent*, then the Tool Agent decides *which tool* — keeps each classifier's job simple. The Copilot's classifier only needs to distinguish broad domains; the Tool Agent's classifier only needs to distinguish between its own three tools. Neither classifier needs to understand the full complexity of the entire platform.

5.4.8 AI Provider Configuration Unification

Before the Copilot was introduced, each agent that called an LLM managed its own API key and endpoint configuration independently. The Tool Agent's DeepSeek service had its own `application.yml` block; the Python inference server read from a separate `ds_config.json` file; the RAG Agent had a third set of environment variables.

The Copilot unified all AI provider configuration under the user-service's `SystemConfigController`. A single admin UI (Settings page) accepts the provider name, base URL, model name, and API key. The key is AES-256 encrypted before storage in MySQL and cached in Redis under `sys:config:ai_provider`. Two REST endpoints serve the configuration:

- `GET /user/config/ai-provider` (public): Returns provider, base URL, and model — the API key is stripped for security. Used by the frontend to display the current model.
- `GET /user/config/ai-provider/internal` (internal, not exposed through the Gateway): Returns the full configuration including the decrypted API key. Called by the Python Flask classifier, the Tool Agent's DeepSeekService, and the RAG Agent. Because this endpoint is not routed through the Gateway, it is only reachable from within the Docker internal network.

With this unification, changing the LLM provider for the entire platform requires updating a single form — no configuration files to edit, no service restarts.

6 System Integration and Deployment

6.1 User Center and RBAC

The user center (user service, port 8081) integrates identity management, access control, department-level data isolation, as well as cross-cutting notification and document services. Table 34 shows the dual-mode login and JWT session management flow.

Table 34: Pseudocode — Dual-Mode Login and JWT Session Management

```
// === Dual-Mode Login ===
ENDPOINT POST /user/login  ->  {username, password, code?}
FUNCTION login(username, password, code): AuthResult
    IF code IS PROVIDED THEN
        // Mode 1 - Email verification code login
        storedCode = redis.get("email:code:" + username)
        IF storedCode IS NULL OR storedCode <> code THEN
            THROW InvalidCodeException()
        END IF
        redis.delete("email:code:" + username)  // single-use
    ELSE
        // Mode 2 - Password login with BCrypt verification
        user = userMapper.selectByUsername(username)
        IF user IS NULL OR NOT BCrypt.matches(password, user.
           password) THEN
            THROW BadCredentialsException()
        END IF
    END IF
    roles = roleMapper.selectByUserId(user.id)
    perms = permissionMapper.selectByRoles(roles)
    token = jwtService.generate({
        sub:      username,
        roles:    roles,
        perms:    perms,
        deptId:   user.deptId,
        clearLv:  user.clearanceLevel,
        exp:      now() + 24h
    })
    redis.set("session:active:" + username, token, {ttl: 30min
       })
    RETURN {token, username, roles, permissions, realName}
END FUNCTION

// === Gateway JWT Validation (JwtAuthGlobalFilter, order =
   -100) ===
FUNCTION filter(exchange, chain):
    IF requestPath IN whitelist THEN  // /login, /register, /
       send-code
        RETURN chain.filter(exchange)
    END IF
    token = extractBearerToken(request)
    claims = jwtService.validateAndParse(token)
    session = redis.get("session:active:" + claims.sub)
    IF session IS NULL THEN THROW SessionExpiredException()
    // Inject downstream headers
    request.mutateHeader("X-User-Name",        claims.sub)
    request.mutateHeader("X-User-Roles",       join(claims.
       roles))
    request.mutateHeader("X-User-Permissions", join(claims.
       perms))
    // Sliding-window renewal (exclude polling endpoints)
    IF NOT requestPath.contains("/notification/unread-count")
       THEN
        redis.expire("session:active:" + claims.sub, 30min)
```

Access Control. Three roles (ROLE_ADMIN, ROLE_DEPT_ADMIN, ROLE_USER) and six granular permissions are used to control access to all protected endpoints. The six departments (Credit Department, Compliance Department, Asset Management Department, Retail Banking Department, Investment Banking Department, and Risk Control Department) provide organizational data scoping. A custom JwtAuthenticationFilter extracts permissions from JWT claims and registers them in Spring Security's SecurityContextHolder. The @PreAuthorize annotation on each controller method can be declared with hasAuthority checks, while the abstraction level's department admin controllers additionally verify that the requested resource's dept_id matches the accessing user's dept_id, enabling cross-cutting department-level data isolation.

6.2 Task Center and Orchestration

The Task Center (task-service, port 8082) is the platform's asynchronous execution backbone. Its role is to accept work from any entry point — the Copilot widget, an agent page, or a future API integration — and ensure that work gets done reliably, with progress visible to the user throughout. Rather than letting each agent manage its own execution lifecycle, the Task Center centralizes queuing, retry, progress reporting, and result persistence.

6.2.1 Task Lifecycle and State Machine

Every piece of work in the platform is modeled as a task record with a four-state lifecycle, shown in Table 35.

State	Meaning	Transition Trigger
INIT	Record created, waiting for a worker thread	Task submitted via REST API or WebSocket
RUNNING	Worker thread picked up the task, calling downstream agent	Executor dequeues the task
SUCCESS	Agent returned a valid result	Downstream agent responds with code 200
FAIL	Execution failed after all retries, or queue was full	Agent error, timeout, or `RejectedExecutionException`

Table 35: Task Lifecycle State Machine

The state machine is linear by design — no complex branching or rollback. If a task fails,

the retry mechanism re-executes from INIT state; it does not attempt to resume from the failure point. This keeps the implementation simple and predictable, which is appropriate for the current single-instance deployment.

6.2.2 Submission, Queuing, and Thread Pool

Tasks enter the system through two channels. The primary channel is `POST /task/submit`, called by the frontend after the Copilot has classified the user's intent. The Task Controller extracts the user's identity from the `X-User-Name` and `X-User-Roles` headers (set by the Gateway's JWT filter) and calls the Task Service. The secondary channel is through the WebSocket handler: when the Copilot widget connects to `/progress?taskId={id}`, it sends a JSON command with the task type and query, which triggers the same submission logic.

Table 36 shows the asynchronous task submission and queuing mechanism.

Table 36: Pseudocode — submitTask() Asynchronous Task Submission and Queuing

On submission, a `TaskRecord` is created in the `task_record` table with status INIT and `attemptCount` set to zero. The record is immediately persisted, so even if the service crashes before execution begins, there is an audit trail of what was requested. The task is then handed to a `ThreadPoolExecutor` configured as follows:

- **Core pool size**: 5 threads (always alive).
- **Maximum pool size**: 10 threads (created under load).
- **Work queue**: `LinkedBlockingQueue` with capacity 100.
- **Rejection policy**: `AbortPolicy` — throws `RejectedExecutionException` when the queue is full and all 10 threads are busy.

The choice of a bounded queue with AbortPolicy is deliberate. In a system without a message broker, an unbounded queue would mask overload by accepting tasks that cannot possibly complete in reasonable time. The 100-slot bound means the system explicitly

refuses work it cannot handle, and the frontend receives an immediate "System busy" response rather than an indefinite hang.

When a `RejectedExecutionException` is caught, the task's status is set directly to FAIL with the message "System busy, please try again later" — the user sees this in the Copilot widget and can retry.

6.2.3 Agent Routing and Execution

Once a worker thread picks up a task, the `executeTaskOnce()` method routes it to the appropriate agent based on `taskType`, shown in Table 37.

taskType	Target Service	Endpoint	Request Body
CODE	code-agent (8084)	`POST /code/query`	`{question: input}`
TOOL / AI	tool-agent (8083)	`POST /tool/execute`	`{toolType: "AI", naturalLanguage: input}`
RAG	rag-agent (8085)	`POST /rag/query`	`{question: input, topK: 5}`

Table 37: Task Type to Agent Routing

All downstream calls use Spring's `RestTemplate` with a 5-second connect timeout and 30-second read timeout. User identity headers (`X-User-Name`, `X-User-Roles`) are forwarded so that each agent can apply its own authorization logic. Service URIs are resolved from environment variables — `CODE_SERVICE_URI`, `TOOL_SERVICE_URI`, `RAG_SERVICE_URI` — with localhost fallbacks for development.

6.2.4 Retry with Exponential Backoff

Network calls to downstream agents can fail for transient reasons: a container restarting, a brief network hiccup, an LLM API rate limit. The retry mechanism in Table 38 handles these gracefully.

Table 38: Pseudocode — executeWithRetry() Exponential Backoff Loop

Each task gets up to 3 execution attempts. Between attempts, the worker sleeps for `1000` $\times$ `attempt` milliseconds — the first retry waits 1 second, the second waits 2 seconds, the third waits 3 seconds. This spacing gives downstream services time to recover from transient failures without hammering them with immediate retries. If all 3 attempts fail, the task is marked FAIL and the error message is stored.

During retries, the WebSocket pushes progress updates ("Retrying... (2/3)") so the user knows the system has not given up. The `attemptCount` field in the database is incremented on each attempt, providing an audit trail for debugging.

6.2.5 Progress Streaming and WebSocket Integration

A detailed discussion of the WebSocket implementation is provided in Section 6.3. From the Task Center's perspective, the key integration point is that progress messages are pushed at domain-meaningful milestones rather than at fixed intervals. For CODE tasks: 10% (analyzing query), 30% (connecting to inference server), 50% (SQL generated, running validation), 80% (executing query), 100% (complete). For TOOL tasks: 10% (analyzing), 25% (intent routed to specific tool), 50% (AI processing), 80% (formatting result), 100%. For RAG tasks: 10% (checking permissions), 45% (vector retrieval), 70% (LLM answer generation), 90% (query logged), 100%.

These milestones are chosen to reflect what the user actually cares about — "Is it still thinking?" "Has it found the data?" "Is it almost done?" — rather than arbitrary percentage ticks.

6.2.6 Current Limitations

The Task Center's design reflects a pragmatic trade-off for a single-instance system. Tasks are queued in memory via `LinkedBlockingQueue`; if the task-service restarts, queued-but-unexecuted tasks are lost (though their INIT records remain in MySQL as orphans). Worker threads block synchronously on downstream HTTP calls via `RestTemplate`, meaning concurrency is capped at the thread pool size of 10. WebSocket session storage is a local `ConcurrentHashMap`, which would not work across multiple instances. These are acceptable constraints for the current deployment scale, and replacing the in-memory queue with a message broker (RabbitMQ or Redis Streams) is a natural next step for production hardening.

Endpoint	Auth	Description
`POST /task/submit`	Headers	Submit an async task; returns task record with ID
`GET /task/{id}`	None	Poll task status and result by ID
`GET /task/list`	Headers	List tasks for the authenticated user (ordered by creation time descending)
`GET /task/list/all`	Admin only	List all tasks in the platform

Table 39: Task Service REST API Endpoints

6.3 WebSocket Real-Time Communication

Long-running AI tasks need progress streaming beyond HTTP request-response. The platform uses WebSocket connections to push task progress from backend to frontend.

The WebSocket task progress architecture, illustrated in the Task Center section above, shows how long-running AI task progress is pushed from the backend TaskProgressWebSocketHandler to the frontend in real time via WebSocket connections proxied through the Spring Cloud Gateway.

Table 40 shows the WebSocket-based task progress streaming mechanism with step-by-step JSON push and frontend rendering.

Table 40: Pseudocode — TaskProgressWebSocketHandler Real-Time Progress Streaming

```
// === Backend: TaskProgressWebSocketHandler ===
// Registered at /tool/ws via WebSocketConfig

CLASS TaskProgressWebSocketHandler EXTENDS TextWebSocketHandler:
    sessions = ConcurrentHashMap<String, WebSocketSession>()

    FUNCTION afterConnectionEstablished(session):
        taskId = extractQueryParam(session.uri, "taskId")
        sessions.put(taskId, session)
        executor.submit(() -> simulateProgress(taskId, session))
    END FUNCTION

    FUNCTION simulateProgress(taskId, session):
        FOR step IN [1..5]:
            sleep(500ms)
            progress = {taskId: taskId, step: step, total: 5,
                        message: "Processing step " + step + "/5"}
            session.sendMessage(new TextMessage(toJSON(progress)))
        END FOR
        sessions.remove(taskId)
    END FUNCTION
END CLASS

// === Frontend: WebSocket Client ===
// Singleton at utils/websocket.ts
FUNCTION connectWebSocket(taskId):
    ws = new WebSocket(BASE_WS_URL + "/tool/ws?taskId=" + taskId)
    ws.onmessage = (event) -> {
        progress = JSON.parse(event.data)
        agentThinking.updateSteps(progress)
        IF progress.step = progress.total THEN
            fetchTaskResult(taskId)  // via AgentThinking.vue
        END IF
    }
END FUNCTION

// Gateway proxy: /ws/** -> tool-agent:8083/tool/ws
// JWT passed as query parameter; upgrade requests whitelisted
```

6.4 Containerized Deployment

Figure 11 shows the Docker Compose service topology, illustrating how each service (Gateway, four backend microservices, frontend, MySQL, Redis, and optionally Milvus with etcd and MinIO) runs in an independent container on an internal Docker network, with only the Gateway port exposed externally.

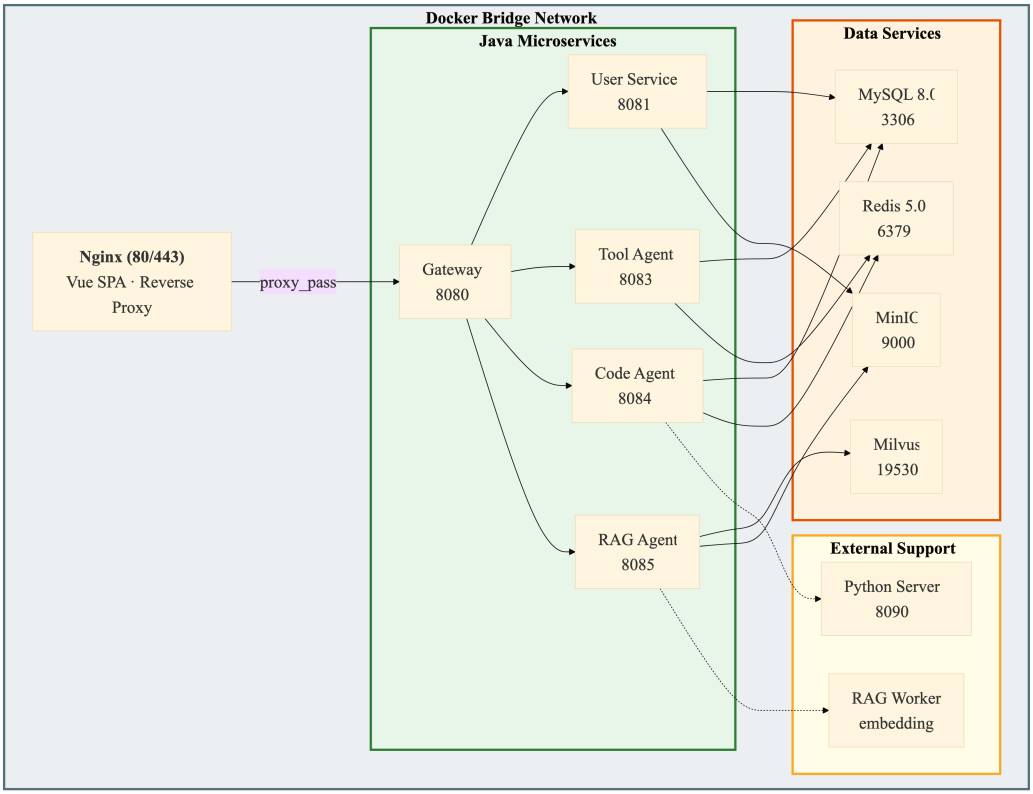

Figure 11: Docker Compose Service Topology

The platform is deployed through Docker Compose. Each service (Gateway, four backend services, frontend, MySQL, Redis, and optionally Milvus with etcd and MinIO) runs in an independent container on an internal bridge network. Only the Gateway (port 8080) is exposed. The frontend Nginx (port 80) serves static Vue 3 SPA files and proxies API and WebSocket requests to the Gateway. MySQL initialization is handled via a mounted SQL dump file. The Python inference server for the Code Agent runs in a separate container with FastAPI.

7 Discussion

7.1 Challenges and Resolutions

Frontend-Backend API Contract Alignment. During early integration, mismatches between the Vue 3 frontend's expected API response structures and the Spring Boot backend's actual response formats caused numerous runtime errors and slowed down development iteration speed. The resolution involved standardizing on a Result<T>generic wrapper (code, msg, data) for all microservice responses. Frontend Axios interceptors were configured to automatically unwrap the data field, so that component code only needs to handle domain data without manually parsing response layers. Additionally, Mock detect fallback is implemented on the backend so that frontend development could proceed independently without waiting for AI APIs to be fully available.

LLM API Integration for AI Intent Routing. The Tool Agent relies on the DeepSeek LLM for natural language intent classification, requiring prompt engineering to produce consistent, machine-parseable output. Several iterations of prompt design were needed. Early versions produced verbose natural language responses that couldn't be reliably parsed, causing the intent routing switch statement to fail or select the wrong handler. A structured output format with explicit categories and parameter schemas was eventually adopted, combined with a mock-detect fallback that returns one of three hardcoded routes when the API is unreachable. Since the iFlytek MaaS platform is compatible with the OpenAI interface, the existing OpenAI library can be used directly.

Copilot Intent Classification. The Copilot adds a second layer of intent classification on top of the Tool Agent's own routing. Getting the classifier to reliably distinguish between CODE, TOOL, and RAG intents required careful prompt design. The key insight was to keep the classification prompt minimal — six category definitions, temperature 0.1, max_tokens 10 — so the LLM returns a single word rather than a justification. Early attempts that asked the model to explain its reasoning produced verbose output that was both slower and harder to parse. The three-tier strategy (blocklist $\rightarrow$ keyword $\rightarrow$ LLM) also emerged from practical iteration: the blocklist and keyword tiers were added after observing that simple greetings and obvious jailbreak attempts were consuming LLM tokens unnecessarily. The CLARIFY intent was a later addition, added when testing showed that

users frequently submitted underspecified requests (“Book a room” without a time, “Plan a route” without a destination) and expected the system to ask for the missing information rather than fail silently.

Amap API Multi-Level Geocoding Fallback. When conducting the initial test of the informal address description, if there is a failure in geocoding, a multi-level rollback strategy needs to be adopted. For example, disambiguate by splitting regions, and use city-level approximate coordinates when detailed addresses cannot be found. After LLM translation, the Chinese navigation instructions are displayed.

Five-Layer SQL Validation Design. Designing a whitelist verification system requires striking a balance between security and availability. An overly strict validator will prevent legitimate analysis queries; an overly permissive one will expose the database to risk. After several rounds testing, the current five-layer architecture was found to be a good balance: operation type restriction (Layer 1) and forbidden keyword blacklist (Layer 2) provide broad security protection at the SQL system level; table and column whitelist (Layer 3 and 4) provide application-level data-scoping control; and query complexity limits (Layer 5) provide fine-grained resource governance and prevent performance-affecting Cartesian product joins. Each layer can be configured separately through the application.yml file, so different environments can adjust strictness according to their actual needs.

ONNX Model Optimization Path. Initially, it was planned to directly use ONNX Runtime for inference. However, this plan was eventually postponed and instead, a Python inference server based on HTTP was used. This happened for two reasons: first, ONNX Runtime’s support is not equally smooth across all model architectures, and the compatibility gap was significant; second, a separate Python server makes it easier to upgrade and replace models—different models can be swapped by changing the Python script only, without requiring changes to the Java service or the Spring context. The cost is additional network latency for the HTTP call, which will be addressed when local ONNX serving is revisited.

7.2 Comparison with Existing Solutions

Compared with cloud-based LLM API services, our platform achieves data sovereignty by processing locally and controlling the external API calls that involve the minimum

amount of data. Only user intent classification queries and geocoding requests go outside; all SQL generation, retrieval, and user data remains within the Docker network.

Compared with general multi-agent frameworks such as AutoGen and MetaGPT, our platform abandons the flexibility of dynamically allocating agent roles and temporary conversations. Instead, it adopts three long-running agents with fixed capabilities (code, RAG, and tool). This design trades theoretical flexibility for operational characteristics that are important in banking: deterministic behavior boundaries, access control at the agent level, and predictable resource usage.

Compared with independent RAG solutions, our integrated architecture enables unified access to three complementary AI capabilities (code generation, knowledge retrieval, and enterprise tool invocation) through a single platform. The trade-off is increased backend complexity; each agent is a separate microservice with its own dependencies, requiring coordinated deployment through Docker Compose and a Gateway for unified entry.

7.3 Project Limitations

Several limitations of the current implementation should be noted. The Code agent's Python HTTP server approach adds network latency and requires a separate Python process; local ONNX Runtime inference would reduce this overhead. The Tool Agent depends on external APIs (DeepSeek, Amap), and the unavailability of these services would degrade functionality to the mock fallback mode. The Task Center uses in-memory queuing via LinkedBlockingQueue rather than a message broker; while adequate for the current scale, this means queued tasks are lost on service restart. Single-instance deployment can handle approximately 50 concurrent users but not multi-node scaling; Kubernetes orchestration is planned as future work. RBAC is intentionally coarse-grained, so fine-grained custom permission design is left to the adopting institution. The Copilot's intent classifier is prompt-dependent — a significant change to the underlying LLM's behavior could require prompt adjustments, and the keyword fallback (Tier 3b) has lower accuracy than the LLM path.

8 Conclusion and Future Work

8.1 Summary of Achievements

This project presents a multi-agent AI platform for banking digital transformation. It addresses issues such as code generation accuracy, domain knowledge reliability, tool invocation robustness, and the data sovereignty demands unique to the banking industry. Through the collaborative work of three specialized agents, a cross-agent intelligent dispatcher, a five-layer defense architecture, and an on-premise Docker Compose deployment model, the platform provides banking personnel with unified AI capabilities accessible through a single natural language interface.

The Code agent implements a natural-language-to-SQL pipeline backed by a Python LLM inference server, combined with a five-layer SQL whitelist validation mechanism enforcing operation-type restrictions, keyword blacklisting, table/column whitelisting, and query complexity limits. The RAG agent combines Milvus vector retrieval with LLM generation, adding document security-level filtering at three stages (pre-retrieval, in-query, and post-retrieval) to ensure policy-compliant knowledge access. The Tool agent implements a dual-path design (programmatic REST endpoints and AI intent routing through the DeepSeek LLM) with mock fallback support, providing meeting room management, Redis-based schedule conflict detection, and Amap-integrated route planning.

Above these three agents, the Copilot serves as a cross-page, multi-agent intelligent dispatcher. Its three-tier intent classification pipeline — a security blocklist, keyword greeting detection, and LLM-based six-category classification — routes natural language queries to the appropriate agent without requiring the user to understand which agent handles which domain. Multi-turn clarification enables the system to gather missing parameters through follow-up questions. A unified AI provider configuration center, accessible through a single admin UI, eliminates the configuration fragmentation that existed when each agent managed its own LLM settings independently.

The system integration layer provides JWT-based authentication with dual-mode login (password and email verification code), three-tier RBAC with department-level data isolation, HITL document access using a notification-based approval system, a Task Center with asynchronous execution, exponential-backoff retry, and WebSocket real-time

progress streaming, and containerized deployment via Docker Compose.

8.2 Future Work

Several directions for future work have been determined. Three directions for model optimization include local ONNX Runtime inference for the Code Agent to reduce HTTP overhead, embedding model evaluation for Chinese financial texts, and LLM hallucination reduction through prompt engineering and citation verification. The Task Center should be upgraded from in-memory queuing to a message broker (RabbitMQ or Redis Streams) for production-grade durability. DevOps enhancements should include Prometheus metrics, Grafana dashboards, distributed tracing, Kubernetes orchestration for multi-node scaling, and automated CI/CD pipelines. The Copilot's intent classifier could benefit from a feedback loop — tracking whether the dispatched agent actually handled the query successfully and using that signal to refine classification prompts. RBAC extensibility should allow adopting institutions to extend the three-tier role model with custom fine-grained permissions. The platform should be extended to support additional enterprise tools (email, HR systems, document management systems) through a pluggable tool registration mechanism, and to support additional database systems beyond MySQL.

References

[1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium, 2018, pp. 3911–3921.

[2] N. Pipitone and G. H. Alami, "LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain," *arXiv preprint arXiv:2408.10343*, 2024.

[3] N. Guha, J. Nyarko, D. E. Ho, C. Ré, A. Chilton, A. Narayana, A. Choksi, *et al.*, "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models," *arXiv preprint arXiv:2308.11462*, 2023.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[5] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Lin, Z. Liu, J. Liu, J. Zhang, Y. Lin, and W. Fan, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *arXiv preprint arXiv:2308.00352*, 2023.

[6] A. Karras, L. Theodorakopoulos, C. Karras, A. Theodoropoulou, I. Kalliampakou, and G. Kalogeratos, "LLMs for Cybersecurity in the Big Data Era: A Comprehensive Review of Applications, Challenges, and Future Directions," *arXiv preprint arXiv:2404.12345*, 2024.

[7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for

Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, 2023.

[11] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.

Appendix A: Database Schema

This appendix documents the complete database schema for the agent_platform database. All tables use InnoDB engine, utf8mb4 character set with utf8mb4_unicode_ci collation. The schema comprises 14 tables across six functional domains.

Table 41: meeting_room — Meeting room resources

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique room identifier
room_name	VARCHAR(50), NOT NULL	Room display name (e.g., “Room 301”)
building	VARCHAR(100)	Building name (e.g., “Building A”)
floor	VARCHAR(20), NOT NULL	Floor identifier (e.g., “3F”)
capacity	INT, NOT NULL	Maximum occupancy
facilities	VARCHAR(200)	Equipment list (Projector, Whiteboard, etc.)
status	INT, DEFAULT 1	1=available, 0=maintenance
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	INT, DEFAULT 0	Logical delete flag

Table 42: meeting_schedule — Booking and schedule records

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique booking identifier
room_id	BIGINT, NOT NULL	FK to meeting_room.id (0 = personal schedule)
booker	VARCHAR(50), NOT NULL	Username of schedule owner
start_time	DATETIME, NOT NULL	Schedule start time
end_time	DATETIME, NOT NULL	Schedule end time
topic	VARCHAR(200)	Optional event description
status	INT, DEFAULT 1	1=active, 0=cancelled
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	INT, DEFAULT 0	Logical delete flag

Table 43: sys_user — User accounts

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(50), UNIQUE, NOT NULL	Login username (email format)
password	VARCHAR(100), NOT NULL	BCrypt-hashed password
real_name	VARCHAR(50)	Display name
role	VARCHAR(20), DEFAULT 'user'	Legacy role description field
status	INT, DEFAULT 1	1=enabled, 0=disabled
dept_id	BIGINT	FK to sys_department
clearance_level	TINYINT, NOT NULL, DEFAULT 1	1=Public, 2=Internal, 3=Confidential
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete (0=normal, 1=deleted)

Seed accounts (all passwords: 123456): admin (ROLE_ADMIN, clearance L3); credit_mgr (ROLE_DEPT_ADMIN, L2, dept_id=1); credit_staff (ROLE_USER, L1, dept_id=1); compliance_mgr (ROLE_DEPT_ADMIN, L2, dept_id=2); compliance_staff (ROLE_USER, L1, dept_id=2).

Table 44: sys_role — System roles

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique role identifier
role_code	VARCHAR(50), UNIQUE, NOT NULL	Role code (ROLE_ADMIN, ROLE_DEPT_ADMIN, ROLE_USER)
role_name	VARCHAR(50), NOT NULL	Human-readable role name
description	VARCHAR(250)	Role description
status	INT, DEFAULT 1	1=enabled, 0=disabled
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete flag

Table 45: sys_permission — Granular permissions

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique permission identifier
perm_code	VARCHAR(100), UNIQUE, NOT NULL	Permission code (e.g., "user:view")
perm_name	VARCHAR(100), NOT NULL	Human-readable permission name
resource_path	VARCHAR(200)	API endpoint path
method	VARCHAR(10), DEFAULT '*'	HTTP method (GET/POST/PUT/DELETE/*)
type	TINYINT, DEFAULT 1	1=menu, 2=API/button
parent_id	BIGINT, DEFAULT 0	Parent permission ID (hierarchical)
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete flag

Seed (6): user:view, user:role, user:status, role:view, task:submit, task:query.

Table 46: sys_user_role & sys_role_permission — Association tables

sys_user_role

Column	Type	Description
user_id	BIGINT, NOT NULL	FK to sys_user.id
role_id	BIGINT, NOT NULL	FK to sys_role.id
PRIMARY KEY		(user_id, role_id)

sys_role_permission

Column	Type	Description
role_id	BIGINT, NOT NULL	FK to sys_role.id
perm_id	BIGINT, NOT NULL	FK to sys_permission.id
PRIMARY KEY		(role_id, perm_id)

Table 47: sys_department — Organizational structure

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique department identifier
dept_name	VARCHAR(100), NOT NULL	Department name
description	VARCHAR(250)	Department description
parent_id	BIGINT, DEFAULT 0	Parent department ID (hierarchical)
status	INT, DEFAULT 1	1=enabled, 0=disabled
create_time	DATETIME	Record creation timestamp

Seed (6): Credit Department, Compliance Department, Asset Management Department, Retail Banking Department, Investment Banking Department, Risk Control Department.

Table 48: sys_document — Knowledge base documents

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique document identifier
title	VARCHAR(100), NOT NULL	Document title
content	TEXT, NOT NULL	Document body (Markdown)
dept_id	BIGINT	FK to sys_department (NULL = global)
security_level	TINYINT, NOT NULL, DEFAULT 1	1=Public, 2=Internal, 3=Confidential
file_type	VARCHAR(20), DEFAULT ‘MARKDOWN’	MARKDOWN / PDF / DOCX / PPT
file_size	BIGINT	Original file size in bytes
minio_object_key	VARCHAR(500)	MinIO object key for uploaded files
parse_status	VARCHAR(20)	PENDING / DONE / FAILED
create_time	DATETIME	Record creation timestamp

Table 49: sys_notification — Multi-type notification system

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique notification identifier
sender_id	BIGINT, NOT NULL	FK to sys_user.id
receiver_id	BIGINT, NOT NULL	FK to sys_user.id
title	VARCHAR(150), NOT NULL	Notification title
content	TEXT, NOT NULL	Notification body
notify_type	VARCHAR(50), DEFAULT 'CHAT'	CHAT / RAG_APPLY / SQL_AUDIT / BUG_REPORT
status	INT, DEFAULT 1	1=Unread, 2=Read/Pending, 3=Approved, 4=Rejected
payload	TEXT	JSON payload for structured metadata
opinion	TEXT	Approval review comments
parent_id	BIGINT	Parent notification ID (threading)
thread_id	BIGINT	Root conversation thread ID
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification
deleted	TINYINT, NOT NULL, DEFAULT 0	Logical delete flag

Table 50: sys_config — System configuration

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique config entry identifier
param_key	VARCHAR(100), UNIQUE, NOT NULL	Configuration key
param_value	VARCHAR(500), NOT NULL	Configuration value
description	VARCHAR(250)	Human-readable description
update_time	DATETIME	Auto-updated on modification

Table 51: task_record — Agent task lifecycle tracking

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique task identifier (starts at 1000)
task_type	VARCHAR(20), NOT NULL	Task type: CODE / RAG / TOOL
status	VARCHAR(20), NOT NULL, DEFAULT 'INIT'	INIT / RUNNING / SUCCESS / FAIL
user_id	BIGINT, NOT NULL	FK to sys_user.id
input	TEXT, NOT NULL	User's natural language prompt
output	TEXT	AI agent output result
error_msg	TEXT	Error details on failure
attempt_count	INT, NOT NULL, DEFAULT 0	Retry count
elapsed_time	INT, DEFAULT 0	Execution time in milliseconds
created_at	DATETIME	Record creation timestamp
updated_at	DATETIME	Auto-updated on modification

Table 52: RAG knowledge base tables

rag_knowledge_base

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique KB identifier
name	VARCHAR(120), NOT NULL	Knowledge base display name
description	VARCHAR(500)	Knowledge base description
owner_username	VARCHAR(100)	Creator or owner username
dept_id	BIGINT	Department scope (NULL = platform-wide)
visibility	VARCHAR(30), NOT NULL, DEFAULT 'DEPARTMENT'	PRIVATE / DEPARTMENT / GLOBAL
security_level	TINYINT, NOT NULL, DEFAULT 1	Default security level for documents
status	VARCHAR(30), NOT NULL, DEFAULT 'ACTIVE'	ACTIVE / ARCHIVED / DELETED
document_count	INT, NOT NULL, DEFAULT 0	Number of documents in this KB
chunk_count	INT, NOT NULL, DEFAULT 0	Total chunk count
create_time	DATETIME	Record creation timestamp
update_time	DATETIME	Auto-updated on modification

rag_source_document

Column	Type	Description
id	BIGINT, PK, AUTO_INCREMENT	Unique source document identifier
kb_id	BIGINT, NOT NULL	FK to rag_knowledge_base.id
sys_document_id	BIGINT	Linked sys_document.id
title	VARCHAR(200), NOT NULL	Display title
original_file_name	VARCHAR(255), NOT NULL	Uploaded file name
file_type	VARCHAR(32)	File extension (pdf, docx, pptx, txt)
mime_type	VARCHAR(120)	Browser-reported MIME type
file_size	BIGINT	File size in bytes
storage_provider	VARCHAR(30), NOT NULL, DEFAULT 'minio'	Storage backend
storage_bucket	VARCHAR(120)	MinIO bucket name
storage_object_key	VARCHAR(500)	MinIO object key
content_hash	VARCHAR(64)	SHA-256 hash of original file bytes
parsed_text	LONGTEXT	Parsed text cache for chunking
status	VARCHAR(30), NOT NULL, DEFAULT 'ACTIVE'	ACTIVE / DELETED
parser_status	VARCHAR(30), NOT NULL, DEFAULT 'UPLOADED'	UPLOADED / PARSED / PARSE_PENDING / PARSE_FAIL
index_status	VARCHAR(30), NOT NULL, DEFAULT 'PENDING'	PENDING / INDEXED / INDEX_FAIL

Appendix B: API Documentation (Selected Endpoints)

POST /api/user/login → {username, password, code?} → {token, username, roles[], permissions[], realName} (JWT signed, 24h expiry; code field optional for verification code login mode)

POST /api/user/send-code → {email} → Sends 6-digit code via Resend (5min TTL, 60s cooldown, daily cap 5)

POST /api/user/register → {username, password, code} → {token, username, roles[], permissions[]}

GET /api/user/me → {id, username, realName, roles[], permissions[], deptId, securityLevel}

PUT /api/user/profile → {realName} → Update display name

PUT /api/user/password → {oldPassword, newPassword} → Change password

User Service (Port 8081) — Authentication:

POST /api/user/login → Authenticate and receive JWT token

POST /api/user/register → Register new account

POST /api/user/send-code → Send email verification code

GET /api/user/me → Get current user profile

PUT /api/user/profile → Update display name

PUT /api/user/password → Change password

Admin (ROLE_ADMIN only):

GET /api/admin/users → [UserResponse[]]; POST /api/admin/user/role → {userId, roleId} → Clears old, assigns new

PUT /api/admin/user/status → {userId, status} → Enable/disable; PUT /api/admin/user/dept → {userId, deptId}

PUT /api/admin/user/security-level → {userId, securityLevel} → Set clearance (1-3)

GET /api/admin/roles → Role[]; GET /api/admin/permissions → Permission[]

Dept Admin (ROLE_DEPT_ADMIN/ROLE_ADMIN):

GET /api/user/dept-admin/members → (deptId? for ADMIN) → [UserResponse[]]

GET /api/user/dept-admin/candidates → Users with null dept_id

POST /api/user/dept-admin/add-members → {userIds[], deptId?} → Batch assign department

Documents:

GET /api/user/document/list → Clearance-filtered [DocumentResponse[]] (accessible boolean, content or "Access Restricted")

POST /api/user/document/create → {title, content, deptId, securityLevel} → ADMIN/DEPT_ADMIN only

PUT /api/user/document/update → {id, title, content, securityLevel} → ADMIN/DEPT_ADMIN (own dept)

DELETE /api/user/document/delete/{id} → ADMIN only

GET /api/user/document/{id} → Document detail (with clearance check)

Notifications:

GET /api/user/notification/list → (status?, notifyType?) → [NotificationResponse[]] enriched with sender/receiver names

GET /api/user/notification/unread-count → Integer count (status=0); POST /api/user/notification/mark-read/{id}

POST /api/user/notification/approve/{id} → replyMessage? → Approve HITL request; POST /api/user/notification/deny/{id} → replyMessage? → Deny

Tool Agent (Port 8083):

POST /api/tool/execute → {toolType, parameters?, naturalLanguage} → AI-routed result (DeepSeek intent classification)

GET /api/tool/meeting-rooms → (startTime?, endTime?, capacity?) → [Room[]] with availability status

POST /api/tool/book-room → {roomId, startTime, endTime, title, attendees} → Booking confirmation

POST /api/tool/check-conflict → {attendees[], startTime, endTime} → {hasConflict, conflictUser, message}

GET /api/tool/schedule/user/{userId} → (startDate?, endDate?) → [Schedule[]]; POST /api/tool/schedule → Create personal schedule

PUT /api/tool/schedule/{id} → Update; DELETE /api/tool/schedule/{id} → Delete

GET /api/tool/route → (origin, destination, waypoints?) → {coordinates[], steps[], distance, duration}

Admin (ROLE_ADMIN):

CRUD /api/tool/admin/meeting-rooms → X-User-Roles header checked; validations: roomName req, floor req, capacity>0

CRUD /api/tool/admin/schedules → X-User-Roles header checked; validations: booker req, startTime<endTime

Code Agent (Port 8084):

POST /api/code/query → {question, database?} → FullResult {success, sql, columns[], rows[], rowCount, elapsedMs, inferenceMethod}

POST /api/code/generate → {question} → {success, sql, inferenceMethod}

POST /api/code/execute → {sql} → {success, columns[], rows[], rowCount, elapsedMs}

GET /api/code/metadata → Cached schema {tables: {}}; POST /api/code/metadata/refresh → Force refresh; GET /api/code/health → Status

Gateway (Port 8080):

Routes: /api/user/**→user-service:8081; /api/tool/**→tool-agent:8083; /api/code/**→code-agent:8084; /ws/**→tool-agent:8083/tool/ws

JwtAuthGlobalFilter (order=-100): Validates JWT on all non-whitelist paths; extracts claims → X-User-* headers; rate limiting: 100 req/s per IP (burst 200)

Appendix C: User Manual (Summary)

Access and Login. Navigate to http://167.172.82.161 and log in with your assigned credentials. Default test accounts: - admin / 123456 (System Administrator, full access, clearance Level 3) - credit_mgr / 123456 (Credit Department Manager, clearance Level 2) - credit_staff / 123456 (Credit Department Staff, clearance Level 1) - compliance_mgr / 123456 (Compliance Department Manager, clearance Level 2) - compliance_staff / 123456 (Compliance Staff, clearance Level 1). New users can register via the Register page with an email-format username and a 6-digit email verification code.

Main Dashboard. The dashboard displays a welcome message with your username, a pending approval alert banner (for ROLE_ADMIN and ROLE_DEPT_ADMIN, showing the count of pending HITL requests with a "Review" button), and a statistics overview with four cards: total users, online users (WebSocket connections), today's agent tasks processed, and pending approval requests.

Tool Agent (/app/tool). The left panel has three tabs: Meeting (room search with date and capacity filters, and booking), Schedule (conflict detection by attendee with a visual timeline grid), and Route (origin/destination input with map rendering via Amap, showing a route polyline and step-by-step guidance). The right panel is the AI Assistant Copilot, a global chat panel.

Code Agent (/app/code). A three-step workflow: (1) enter a natural language query or select a quick template, (2) review the generated SQL in a code editor with model information, and (3) execute and view results in a data table with pagination. The result table shows columns, row count, the generated SQL, and execution time. A security rejection message is displayed if the SQL fails any validation layer.

Documents (/app/dept-docs). Two tabs: System Manuals (global documents) and Department Assets (clearance-controlled). Click a document to open a Zen Reader with markdown rendering and an auto-generated table of contents. Documents beyond your clearance level show "Access Restricted" with an "Apply for Access" button that submits a RAG_APPLY notification.

Messages (/app/notification). A three-column layout: left sidebar with filters (All, Un-

read, Pending Approvals, Sent), middle message list, and right detail panel. Type-specific renderers handle RAG_APPLY approvals (with approve/deny buttons), SQL_AUDIT records (with SQL code blocks), and BUG_REPORT feedback.

Register (/register). New users can create an account with an email-format username and a 6-digit email verification code. The registration form requires the user to enter their email address, request a verification code (sent to their inbox, valid for 5 minutes), set a password, and confirm the code. Server-side rate limiting enforces a 60-second cooldown between code requests and a maximum of 5 code requests per email address per day.

Administration (/app/admin/users, /app/admin/resources, /app/admin/my-dept). User Management provides a searchable table of all users with filters for department, clearance level, and role. Admins can assign roles, toggle account status, set clearance levels, and manage department membership. Resource Management lets admins manage meeting rooms and view all schedules. My Department (for ROLE_DEPT_ADMIN) shows members within their own department with the ability to view and manage membership.

Settings (/app/settings). Two configuration dialogs: Profile (update your display name through PUT /user/profile) and Security (change your password with current password verification, a new password must be at least 3 characters via PUT /user/password).

Declaration of Individual Contributions

In accordance with the University of Hong Kong's guidelines for group project reports, this section details the specific contributions of each team member.

Lin Chuanbin:

In the Agent Platform project, I was responsible for developing three core modules from scratch: the Tool Agent microservice, the User Service microservice, and the full-stack implementation of the initial front-end version, delivering a total of 30+ core files. Below are my specific responsibilities:

1 Tool Agent Module

The Tool Agent is the core intelligent agent of the system, built on Spring Boot, integrating the DeepSeek large language model to achieve natural language understanding and intent recognition. It covers five subsystems: AI intent routing, meeting room management, schedule conflict detection, intelligent route planning, and WebSocket real-time communication.

1.1 System Architecture and AI Intent Recognition

I designed a layered architecture for the Tool Agent, clearly defining the Controller layer, Service layer, external service layer, and data access layer. The Controller layer provides a unified RESTful API entry point; the Service layer is responsible for core business functions such as intent routing, meeting room management, and schedule conflict detection; the external service layer encapsulates DeepSeek AI calls and the Gaode Map API; and the data layer implements database access through MyBatis-Plus.

The AI intent recognition module is the intelligent brain of the Tool Agent. I encapsulated the communication logic with the DeepSeek API, using a RESTful approach and supporting a dual-role dialogue mode of system prompts and user content. The system defines three intent types (route planning, meeting room search, and schedule conflict detection). After DeepSeek analyzes the intent, it automatically routes the request to the corresponding business processing method.

Considering the potential unavailability of AI services in a production environment, I implemented a robust fallback mechanism: when a DeepSeek call fails, the system returns

preset demo data based on the tool type, ensuring the front-end always displays correctly and the user experience is not affected by AI service failures.

1.2 Tool Agent Module

The meeting room module contains two core data models: meeting room entities and schedule booking entities. I implemented time period overlap detection logic, returning complete data including availability status and a list of devices. The booking function first checks for conflicts; if the time period is already booked, it returns a failure.

For AI intelligent matching, the system calls DeepSeek to analyze the user's meeting room needs, extracting information such as date, number of people, and equipment. The returned results include an AI matching score and recommendation reasons, helping users quickly find the most suitable meeting room.

1.3 Schedule Conflict Detection (High-Performance Redis Implementation)

Schedule data is stored in a Redis ordered set, using user ID as the key and timestamp as the score, achieving range queries with O(log N) time complexity. I designed three core interfaces: range query conflict detection, schedule data storage, and record deletion by event ID.

Timestamp calculation uses the East 8 time zone to ensure consistency with domestic business time.

1.4 Intelligent Route Planning (Gaode API Three-Level Degradation)

I encapsulated the calls to the Gaode Map REST API, mainly using three interfaces: geocoding, POI search, and driving route planning. To address the address resolution success rate issue, I designed a three-level degradation strategy: first, attempt direct geocoding; if it fails and the address does not contain a city prefix, automatically append the "Beijing" prefix and retry (this was later modified); finally, use POI text search as a fallback, significantly improving the address resolution success rate.

The system formats the raw data returned by Gaode Maps in a user-friendly way: distance is automatically converted to "km/m" units, duration to "hours/minutes" units, and traffic conditions are automatically classified into three levels: "smooth/slow/congested" based on average speed.

1.5 WebSocket Real-time Task Progress Push

The system provides a real-time task progress push service via WebSocket, supporting the establishment of independent connections by task ID. A concurrent and secure session management mechanism is used to manage all active connections, simulating a complete processing flow (parsing natural language → querying the database → AI model inference → organizing results), with progress updates pushed at fixed intervals for each stage. A thread pool is used to manage task execution, supporting a maximum of 10 concurrent tasks.

2 User Service Module

The User Service is the system's user authentication and permission management microservice, built on Spring Boot + Spring Security, using JWT to implement stateless identity authentication, supporting encrypted password storage and interface-level permission control. Yao Jiacheng and Xia Xin subsequently made some improvements and developments on this basis.

2.1 System Architecture Design

The User Service adopts a layered architecture, including a Controller layer, a Service layer, a data access layer, and an entity/DTO layer. Global security control is implemented through Spring Security configuration, including CSRF disabling, stateless session management, allowing login interfaces, and authentication protection for other interfaces.

2.2 JWT Authentication and Security Module

I encapsulated complete lifecycle management of JWTs, including token generation, parsing, and verification. The HMAC-SHA256 signature algorithm is used, with the key and expiration time injected from the configuration file for flexible configuration management. The generation method encodes the username and role information into the token; the parsing method verifies the token signature; the verification method catches all exceptions and returns a boolean value to ensure the security of the caller.

The security policy is configured with complete Spring Security protection: disabling CSRF (using JWT instead of Session in a front-end/back-end separation architecture), setting up stateless session management, allowing login interfaces, and requiring authentication for all other requests. Passwords are stored using BCrypt with one-way hash encryption.

2.3 User Login and Permission Management

The login process implements a complete four-step security verification: querying users by username, checking user status, BCrypt password matching verification, generating a token, and encapsulating the response for return.

I designed standardized request and response DTOs, using annotations to implement parameter validation to ensure that usernames and passwords are not empty. I also defined a unified generic response encapsulation class containing three fields: status code, message, and data, providing static factory methods for success and failure to ensure consistent return formats across all microservices.

3 Front-end Development (Initial Version)

I was responsible for building the initial version of the front-end project and implementing the core pages, providing the basic framework for subsequent iterative development by team members (later, Yao Jiacheng and Xia Xin proposed many excellent new ideas and made many significant improvements). My front-end uses the Vue 3 + TypeScript + Vite technology stack, combined with the Element Plus UI component library and Pinia state management.

3.1 Front-end Technology Architecture and Engineering

The project uses Vue 3's composable API to organize code logic, TypeScript for type safety, and Vite as the build tool for rapid development and hot updates. I configured path aliases to significantly improve code maintainability. I also configured API proxies and WebSocket proxies to solve cross-domain issues in the development environment. Element Plus uses on-demand plugin loading to import components as needed, reducing the package size.

3.2 Core Page Implementation (Initial Version)

Login Page: Uses a centered card layout with a gradient background for a modern feel. A form component is used for username and password input, with validation rules ensuring input integrity. The login logic calls a state management method; upon successful login, the token is persisted and the user is redirected to the homepage.

Main Layout Page: Uses a classic sidebar + top navigation + main content area layout. The sidebar implements a multi-level menu, and the top navigation displays current user information and logout functionality. The main content area uses transition animations to switch pages.

The tool call page adopts a left-right split layout. The left side is the function operation area (three tabs: meeting room search, schedule conflict detection, and route planning), and the right side is the AI assistant panel (natural language input, WebSocket real-time progress, and AI analysis result display). The map component encapsulates the initialization, route drawing, origin and destination marking, and view adaptation functions of Gaode Map, and uses a dynamic script loading method to import the map API.

Note: The above front-end components and pages are the initial version. Subsequent versions were refactored and expanded by Yao Jiacheng and Xia Xin, including a multi-tab system, responsive mobile adaptation, a global Copilot component, and a unified UI design system—all production-grade improvements.

3.3 State Management and HTTP Encapsulation

I designed a user state management module, defined using Pinia's Setup Store pattern, including responsive states such as tokens and user information, as well as Action methods such as login, logout, and retrieving user information. Upon successful login, the token is persisted to local storage; upon logout, the state is cleared and the user is redirected back to the login page. A task state management module was also designed, supporting adding tasks and updating task states.

The HTTP request encapsulation uses Axios to implement a unified request instance, configured with a relatively long timeout (to adapt to AI inference time), a basic path, and JSON content type. The request interceptor automatically reads the token and adds it to the request header. The response interceptor handles backend responses uniformly: returning data on success and displaying an error message on failure; unauthorized status triggers login expiration confirmation, and the user is automatically logged out and redirected to the login page after confirmation.

3.4 WebSocket Client and Task Orchestration (Initial Version)

I encapsulated a WebSocket client class, providing core methods such as connect, send, close, and event listeners. It supports an automatic reconnection mechanism (up to 3 times, 2-second interval), and achieves loosely coupled message processing through an event listener pattern. A singleton pattern is used to ensure global sharing of the same connection.

Simultaneously, I wrote the initial gateway center orchestration logic, implementing static

Agent routing and WebSocket progress updates, providing the infrastructure for the subsequent implementation of asynchronous execution pipelines, failover, and retry mechanisms.

Yao Jiacheng:

I served as the backend architecture lead and, over the course of the project, took on full-stack development, system integration, security, and DevOps. Much of my work involved extending the initial implementations contributed by other members into production-grade features.

1. Backend

1.1 RBAC authentication & authorization

Building on the initial user-service skeleton (basic JWT login with a single admin/user role) created by Lin Chuanbin, I extended the platform's security model into a multi-layer permission architecture:

- Designed five RBAC tables supporting three role types (admin, department admin, regular user) with fine-grained permissions pre-seeded in the database. - Replaced the original single-role JWT filter with a custom JwtAuthenticationFilter that extracts roles and permissions from JWT claims and registers them in Spring Security, enabling declarative @PreAuthorize checks. - Built admin and department-admin controllers covering user CRUD, role assignment, department assignment, clearance level management, and account lifecycle operations. - Introduced multi-level department trees with automatic department-scoped data isolation in all data-access controllers. - Implemented three-tier document clearance enforcement with department scoping, clearance level comparison, and HITL override via approved access requests.

1.2 API gateway, session management & unified response format

I built the platform's unified API gateway layer and standardized the cross-service response format:

- Built a custom Spring Cloud Gateway global filter with route rules covering all microservices and WebSocket endpoints. - Implemented single-session enforcement via Redis (new logins invalidate old sessions), sliding-window TTL renewal with polling endpoint exemption, and dynamic admin-configurable session timeout. - Propagated user identity

through HTTP headers so downstream services operate without holding JWT_SECRET. - Defined the Result<T>generic wrapper standard, deployed identical DTOs across all four microservices, and wrote the frontend Axios interceptor with automatic unwrapping.

1.3 Notification & HITL approval system

I independently designed and implemented the platform's notification and approval subsystem:

- Designed the notification table with threaded messaging support (parent_ id/thread_-id) and a five-state state machine spanning unread, read, pending, approved, and rejected statuses. - Built the notification controller with endpoints for sending, filtered listing, read marking, approve/deny with audit trail, and threaded conversation view.

1.4 Task center orchestration

I designed and implemented the task center, a centralized orchestration layer that tracks and manages all Agent workflows across the platform. When a user submits a request through any Agent, the task center persists it as a task record, queues it for asynchronous execution, and pushes real-time progress updates to the frontend.

- Built the task-service microservice with an async execution pipeline. - Implemented routing by task type to the corresponding Agent, with progress pushed in three stages to the frontend. - Designed fault-tolerance mechanisms including connection timeout, read timeout, up to 3 automatic retries, and graceful queue overflow rejection to prevent system overload.

1.5 Email verification system

I replaced the initial QQ SMTP implementation (originally contributed by Xia Xin) with a production-grade solution after identifying reliability and security limitations:

- Migrated from QQ SMTP to the Resend HTTP API with a redesigned HTML email template. - Hardened security with brute-force protection against verification code guessing and a three-tier rate limiting system (per-email cooldown, per-IP hourly cap, per-email daily cap).

1.6 Copilot backend & AI provider unification

I built the backend for the global Copilot assistant, which generalized the AI chat panel originally confined to Lin Chuanbin's Tool Agent page into a cross-page, multi-Agent in-

telligent dispatcher that sits above all three agents as a unified natural language entry point. The Copilot's Python Flask intent classifier implements a three-tier classification pipeline — security blocklist, keyword greeting match, and LLM-based six-category classification (CODE, TOOL, RAG, CHAT, CLARIFY, SETTINGS) — complemented by multi-turn clarification, cross-page result delivery via browser custom events, and WebSocket-based progress streaming for long-running agent calls. I unified all AI provider configurations across the platform into a single admin-managed configuration center with AES-256 encryption, eliminating the fragmentation that existed when each agent managed its own LLM settings independently.

1.7 Document management & MinIO storage

I worked with Huang Siyuan to build the document storage and management backend, which serves both the document library and her RAG Agent:

- Built a MinIO storage service with auto bucket creation, admin-configurable upload size limits, and support for PDF, DOCX, PPT, and Markdown files. - Extended the document table with file metadata and parse status columns to support RAG document parsing. - Implemented document access request approval routing to department managers with fallback to system admin.

2. Frontend

2.1 Layout, navigation & multi-tab system

I rebuilt the frontend shell from the initial scaffold created by Lin Chuanbin into a production-grade application framework:

- Rebuilt the main layout with a collapsible sidebar, multi-tab top bar, responsive mobile layout with off-screen drawer, and role-based dynamic menu visibility. - Built a browser-style multi-tab system that preserves page state when users switch between tabs — form inputs, filters, and scroll positions remain intact — preventing task interruption, a common issue in enterprise workflows. - Implemented a five-layer global navigation guard covering silent refresh recovery, authentication redirect, bounce-back, single-role check, and multi-role check.

2.2 UI design system

I established a minimalist visual design language across the entire platform, replacing the

default Element Plus styling inherited from the initial scaffold:

- Globally replaced the default blue theme with a grayscale color system, removing accent strips and unifying border-radius, sizing, and spacing. - Redesigned notifications with macOS-style frosted glass and custom cubic-bezier slide-in animation.

2.3 Tool Agent UI rebuild

I redesigned all three Tool Agent interfaces originally built by Lin Chuanbin, splitting the monolithic 600-line file into independent sub-components and removing the fixed AI chat panel in favor of an inline interaction model:

- Meeting Agent: rebuilt search as a horizontal inline bar with a responsive card grid, SVG icons, equipment tags, and a booking dialog. - Schedule Agent: unified two separate forms into a single filter panel and replaced manual user ID input with a filterable multi-select dropdown. - Route Agent: redesigned the map from conditional rendering (hidden when no results) to a persistent full-width display. Added a floating control panel and clickable route segments that show detailed navigation steps.

3. DevOps

I set up the full deployment pipeline for the platform, bringing all three Agent services — Zhang Zhaoming's Code Agent, Huang Siyuan's RAG Agent, and Lin Chuanbin's initial Tool Agent — onto a single DigitalOcean Droplet:

- Wrote the docker-compose.yml that starts all 9 services with proper startup order, health checks, and data persistence. - Created Dockerfiles for each Spring Boot service. - Deployed a DigitalOcean Droplet. To keep the Maven build from running out of memory on a small instance, I compiled the JARs locally and copied them over. - Set up Nginx as the only public-facing entry point, handling SPA routing, API/WebSocket proxying, and static file caching. - Registered the domain bankagent.online through Namecheap and configured Let's Encrypt for HTTPS access.

Huang Siyuan:

I was responsible for building the RAG Agent into a complete enterprise document Q&A subsystem. I created a standalone rag-agent microservice on port 8085, integrated it with the Spring Cloud Gateway via the new /api/rag/** route, designed three

new database tables (rag_document_chunk, rag_milvus_vector, rag_query_log) and integrated Milvus as the vector database with HNSW/IVF_FLAT index for approximate nearest neighbor retrieval. I implemented document chunking with structure-aware splitting and configurable overlap, embedding via external API and local ONNX models, and permission-enforced knowledge retrieval through pre-retrieval scope computation and post-retrieval re-verification. I defined five REST API endpoints (POST /rag/query, POST /rag/index/rebuild, POST /rag/index/document/{id}, DELETE /rag/index/document/{id}, GET /rag/health) protected by Gateway-level JWT validation. I also built the frontend RAG workbench page with a split-pane layout: left-side question input with example prompts, right-side answer panel with cited source documents, and a bottom toolbar with permission filter indicators and index management controls.

Xia Xin:

I contributed across backend security, frontend internationalization, documentation standardization, authentication, multilingual support, and visual communication of system architecture.

1. Backend

1.1 Authentication

I built the platform's initial Spring Security layer with stateless JWT tokens (HMAC-SHA256) and BCrypt password encoding to separate admin and regular user access. I configured the security filter chain to expose only the login and registration endpoints publicly while requiring authentication for all other routes, and defined the foundational Result<T> response wrapper and DTOs to ensure consistent error handling and data exchange across the codebase. This part was upgraded by Yao Jiacheng.

1.2 Email Verification Integration Building on the existing team codebase, I integrated the QQ Mail SMTP service to deliver verification codes during registration. I stored codes in Redis with a 5-minute TTL, enforced a 60-second per-email send rate limit to prevent abuse, and wired the code verification step directly into the registration flow, completing the end-to-end sign-up process. This part has been upgraded into Resend HTTP API by Yao Jiacheng.

2. Frontend & Internationalization

2.1 Multi-Language i18n Support I adapted all business and UI components to support three languages (Simplified Chinese, Traditional Chinese, and English) with dynamic switching through Vue3 and Element Plus i18n. This required systematically reorganizing component-level text, validation messages, and document metadata into structured locale files, ensuring the entire frontend remained maintainable and language-agnostic. Also, I designed a initial version of Code Agent's frontend, which has been upgraded into a better version by Yao Jiacheng.

3. Documentation & Visualization Standardization

3.1 System Diagrams Redesign I designed the first version of all figures, and after team discussion, I redrew the original dense, small-font diagrams into clean, wide-format vector graphics with a unified visual style. Every diagram maintained readability and visual hierarchy, ensuring clear communication of complex architectural concepts. A total of 12 figures were created and refined.

3.2 Final Report & Table Standardization

I was responsible for synchronizing the final report with the latest codebase. I converted all seven data tables into the three-line table format (1.5 pt top and bottom rules, a 0.75 pt rule below the header, no vertical or interior horizontal lines) and suggested moving long pseudocode blocks to standalone tables—both to maintain formatting consistency and to communicate algorithms more clearly to readers.

Zhang Zhaoming:

I designed and implemented the Code Agent module — the natural-language-to-SQL engine that translates user questions into MySQL queries, validates them through a five-layer security whitelist, and executes them safely against the database. The module comprises a dedicated Spring Boot microservice and a Python LLM inference bridge. I also conducted model-level experimentation, fine-tuning two T5 variants before converging on the final LLM API-based approach.

1. Code Agent Microservice Architecture

I built the code-agent microservice (port 8084) from scratch as a standalone Spring Boot 3.2 service.

Designed the service with a clean separation of concerns: CodeAgentController (REST layer, 6 endpoints), CodeGenerationService (interface with pluggable implementations), SqlValidationService (five-layer whitelist engine), SqlExecutionService (JdbcTemplate-based safe execution), MetadataCacheService (information_schema → Redis), and MetadataCacheManager (lifecycle management).

Implemented the OnnxCodeGenerationService as an HTTP proxy to the Python LLM inference server (port 8090), using java.net.http.HttpClient with 10-second connect timeout and 60-second response timeout. The contract is POST question → returns sql, method JSON.

Built the CodeAgentProperties configuration class with @ConfigurationProperties(prefix = ”code-agent”), supporting three config groups (ONNX inference, SQL whitelist, metadata cache) all overridable via environment variables for seamless Dev/Prod switching.

2. Five-Layer SQL Whitelist Validation Engine

I designed and implemented a defense-in-depth SQL validation pipeline (SqlValidationService) that blocks injection and unauthorized queries at five sequential layers, each with immediate rejection on failure:

Layer 1 — Operation Type: Enforces SELECT-only. Any non-SELECT statement is rejected before further processing, preventing write operations at the earliest stage.

Layer 2 — Forbidden Keywords: Scans SQL with word-boundary regular expressions (`\bDROP\b`, `\bDELETE\b`, `\bINSERT\b`, `\bUPDATE\b`, `\bALTER\b`, `\bTRUNCATE\b`, `\bCREATE\b`, `\bEXEC\b`, `\bUNION\b`, `\bINTO\b`, `\bLOAD_ FILE\b`, `\bOUTFILE\b`, `\bDUMPFILE\b`). The `\b` boundary prevents false positives — for example, `customer_no` is not flagged for containing the substring IN.

Layer 3 — Table Whitelist: Extracts all tables referenced in FROM and JOIN clauses via regex and validates each against the cached information_schema table list. Unknown tables are rejected with the allowed set shown in the error message.

Layer 4 — Column Whitelist: For non-SELECT * queries, extracts every column from the SELECT clause and validates each against the actual column definitions of the referenced tables — supporting JOIN queries by merging all referenced tables’ column sets. Aggregate functions and expressions are intelligently skipped.

Layer 5 — Complexity Limits: Caps queries at a maximum of 3 joined tables and 10 conditions (counting AND/OR occurrences + 1 base). This prevents resource exhaustion from overly complex LLM-generated queries.

All layer parameters are configurable through code-agent.whitelist.* in application.yml.

3. Metadata Caching & Execution Infrastructure

I built the metadata caching layer and safe SQL execution service:

Implemented MetadataCacheService that queries MySQL information_ schema (using JDBC directly, not MyBatis) to extract table names, column names, data types, and comments — caching them in Redis as a JSON Hash map keyed by table name with a 1-hour TTL. When Redis is unavailable, the service falls back to direct MySQL queries, ensuring zero-downtime operation.

Built MetadataCacheManager with an @EventListener(ApplicationReadyEvent.class) for startup cache warm-up and a @Scheduled(fixedRate = 30 * 60 * 1000) for periodic refresh every 30 minutes.

Implemented SqlExecutionService with defense-in-depth: before executing any SQL, it re-validates through the full five-layer pipeline, catching edge cases where SQL might have been tampered with between generation and execution. Only then does it run via JdbcTemplate.queryForList(sql) and return results with column names and data rows.

Built the generateAndExecute one-shot method that chains generation → validation → execution into a single FullResult response.

4. REST API Endpoints

I designed and implemented all six REST endpoints for the Code Agent, centered around the Human-in-the-Loop (HITL) pattern that separates generation from execution:

POST /code/query is the one-shot endpoint: it accepts a natural language question, calls the LLM inference service to generate SQL, runs the five-layer validation, executes the query, and returns the full result including column headers, data rows, row count, and elapsed time.

POST /code/generate handles SQL generation only — it takes a question and returns the LLM-generated SQL with inference metadata, without executing anything. This enables the frontend to display the generated SQL for human review before it touches the database.

POST /code/execute accepts a reviewed (and potentially human-edited) SQL string, re-validates it through the full whitelist pipeline, and executes it — completing the supervised HITL workflow.

GET /code/metadata returns the currently cached table metadata from Redis or MySQL fallback.

POST /code/metadata/refresh forces an immediate refresh of the metadata cache from information_schema.

GET /code/health returns the service status and active inference method for operational monitoring.

5. Model-Level Experimentation: Fine-Tuned T5 vs. LLM API

Before settling on the final architecture, I conducted practical experiments comparing local fine-tuned models against cloud LLM APIs for the text-to-SQL task:

Fine-tuned two T5 variants on our banking-domain dataset, deploying them via ONNX Runtime for local inference. The goal was to evaluate whether a small, domain-specific model could match general-purpose LLMs on our constrained schema.

Through systematic comparison, I found that the fine-tuned T5 models consistently underperformed the LLM API approach — even with careful training, the smaller models struggled with schema grounding, complex JOIN logic, and edge cases that a general-purpose LLM handled naturally when given well-structured metadata in the prompt.

This led to the key architectural decision: rather than investing further in local model optimization, I adopted the LLM API + rich metadata prompt strategy, where the cached information_schema context (table names, column definitions, data types, example values) is injected into every generation request. This approach proved both more accurate and more maintainable, as schema changes require no model retraining — only a metadata cache refresh.

6. Test Dataset & Banking Domain

I authored the banking_data.sql test dataset with three relational tables: bank_customer (5 rows, risk levels LOW, MEDIUM, and HIGH), bank_account (7 rows, account types SAVINGS, CHECKING, and FIXED), and bank_transaction (10 rows, transaction types DEPOSIT, WITHDRAW, and TRANSFER). These cover diverse NL2SQL scenarios from

simple lookups to multi-table JOIN aggregations, providing a realistic testbed for the generation, validation, and execution pipeline.